Министерство науки и высшего образования Российской Федерации

ФБГОУ ВО «Кубанский государственный технологический университет»

(ФГБОУ ВО «КубГТУ»)

Факультет информационных технологий и кибербезопасности

Кафедра информационных систем и программирования

Направление подготовки 09.03.04 программная инженерия

Профиль проектирование и разработка программного обеспечения

КУРСОВОЙ ПРОЕКТ

по дисциплине «Технологии разработки программного обеспечения»

на тему: Разработка программного обеспечения для сведений о десяти нападающих хоккейных команд города

Выполнил студент Лисненко Мария Максимовна курса 2 группы 24-КБ-ПР2

Допущен к защите 17.11.25г

Руководитель (нормоконтролёр) проекта доц. К.А. Ковалева

Защищен 24.11.25г Оценка 5 (отл)

Члены комиссии доц. В.А. Мурлина

ст. преп. Ю.С. Вонарх

Краснодар

2025г.

ФБГОУ ВО «Кубанский государственный технологический университет»
(ФГБОУ ВО «КубГТУ»)

Факультет информационных технологий и кибербезопасности
Кафедра информационных систем и программирования
Направление подготовки 09.03.04 программная инженерия
Профиль проектирование и разработка программного обеспечения

УТВЕРЖДАЮ
Зав. кафедрой ИСП
доц. М.В. Янаева
10. 09. 2025 г.

ЗАДАНИЕ
на курсовой проект

Студенту Лисненко Марии Максимовне курса 2 группы 24-КБ-ПР2
Тема проекта: Разработка программного обеспечения для сведений о десяти нападающих хоккейных команд города
(утверждена указанием директора института № 24 (от 09. 09. 2025 г.)
План проекта:
1. Описание предметной области
2. Проектирование
3. Реализация

Объем проекта:
а) пояснительная записка 57 с.
б) иллюстративная часть — лист(-ов).

Рекомендуемая литература:
1. Зубкова, Т. М. Технология разработки программного обеспечения : учебное пособие / Т. М. Зубкова. — Санкт-Петербург : Лань, 2022. — 324 с. — ISBN 978-5-8114-3842-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/206882
2. Федькова Н.А. Современные технологии разработки про-граммного обеспечения: уч.-метод. пособие – Брянск : Брянский ГАУ, 2022. — 58 с. — URL: https://e.lanbook.com/book/305087 (дата обраще-ния: 13.06.2024).

Срок выполнения работы: с "09" 09.2025 по "12" 12. 2025 г.
Срок защиты: "26" 12. 2025 г.
Дата выдачи задания: "09" 09. 2025 г.
Дата сдачи проекта на кафедру: "12" 12. 2025 г.

Руководитель проекта доц. К.А. Ковалева
Задание принял студент М.М. Лисненко

Вариант 11.

1. Сведения о 10 нападающих хоккеистах моего города.
2. Структура таблицы:

- фамилия хоккеиста;
- номер игрока;
- амплуа (нападающий);
- дата рождения;
- дата подписания контракта;
- фамилия тренера;
- зарплата.

2. Предусмотреть возможность добавления записей в таблицу, редактирования, удаления.
3. Определить:

- количество хоккеистов, тренирующихся у тренера с заданной фамилией;
- зарплату наиболее высокооплачиваемого хоккеиста;
- среднюю зарплату хоккеистов.

4. Создать таблицу, которая содержит фамилию хоккеиста, номер игрока и продолжительность контракта.
5. Сортировать данные в таблице по заданному столбцу.
6. Изменить сведения о хоккеистах, подписавших контракт позже заданной даты.
7. Удалить информацию о хоккеисте с заданной фамилией.

Реферат

Пояснительная записка курсового проекта содержит: 77 с., 25 рис., 7 табл., 2 листинга, 5 источника, 2 приложения.

ПРОЕКТИРОВАНИЕ, РАЗРАБОТКА, UML-ДИАГРАММЫ, АННОТАЦИЯ IDEF0, IDEF3, DFD, ПРОГРАММИРОВАНИЕ, БАЗА ДАННЫХ, C#, SQL, ПРИЛОЖЕНИЕ, ИНТЕРФЕЙС, DRAW IO, VISUAL STUDIO, ENTITY FRAMEWORK, METROFRAMEWORK

Цель работы — разработать программное обеспечение для управления сведениями о хоккеистах-нападающих, используя полученные теоретические знания и практические навыки в области технологий разработки.

Объект исследования — процесс управления информацией о хоккеистах-нападающих.

Предмет исследования — разработка программного обеспечения для автоматизации учета хоккеистов, включая функции добавления, редактирования и удаления записей, а также анализа данных о хоккеистах.

Программное обеспечение создано в виртуальной среде программирования (IDE) Visual Studio на языке программирования C# и языке запросов SQL. Иллюстрации и схемы выполнены в All Fusion Process Modeler R7, Draw IO и Microsoft Excel 2022.

Содержание

Введение 6
Нормативные ссылки 6
1. Жизненный цикл программного продукта 9
1.1 Анализ задачи 9
1.2 Разработка технического задания на проектируемую систему 9
1.3 Управление проектом 11
2 Модульное проектирование и разработка 21
2.1 Обоснование выбора среды реализации 21
2.2 Построение контекстной диаграммы в нотации IDEF0 22
2.3 Построение диаграммы декомпозиции в нотации IDEF0 25
2.4 Определение компонентов для построения диаграммы потоков данных (DFD) 30
2.5 Определение компонентов, их связывание при построении модели IDEF3
3 Структурное проектирование и разработка 32
3.1 Построение диаграммы состояний 32
3.2 Построение диаграммы последовательности. Построение диаграммы кооперации 33
3.3 Построение диаграммы компонентов 35
4 Объектно-ориентированное проектирование и разработка 36
4.1 Построение диаграммы Сущность-связь 36
4.2 Построение диаграммы вариантов использования 36
4.3 Построение диаграммы деятельности 37
4.4 Построение диаграммы классов 39
5 Инструментальные средства тестирования, проектирования и разработки программного обеспечения 40
5.1 Разработка пользовательского интерфейса 40
5.2 Разработка информационной базы проекта 44
5.3. Проведение тестирования программной системы, выявление, анализ и исправление ошибок 48
5.4 Разработка структуры документации, приложений к программному обеспечению. Формирование документации в соответствии с нормативными требованиями 49
Заключение 51
Список использованных источников 52
Приложения А ..... **Error! Bookmark not defined.**

Введение

В условиях современного спортивного рынка, где эффективное управление командой становится важным фактором успеха, учет сведений о хоккеистах-нападающих играет ключевую роль в успешности клуба. С увеличением количества игроков и объемов данных возникает необходимость в эффективной системе учета, которая позволит не только хранить информацию о хоккеистах, но и оперативно обрабатывать ее, что в свою очередь способствует повышению качества управления командой и оптимизации рабочего процесса.

Целью реализации данного курсового проекта является разработка программного обеспечения, которое будет обеспечивать учет сведений о хоккеистах-нападающих. Система будет включать в себя таблицы с информацией о хоккеистах, тренерах и других важных аспектах, необходимых для эффективного управления командой. Программное обеспечение позволит автоматизировать процессы обработки данных о хоккеистах, что значительно упростит работу сотрудников и повысит качество управления. Кроме того, система будет включать функции аналитики, позволяющие отслеживать производительность тренеров и выявлять тенденции в составе команды, что поможет в оптимизации спортивных процессов.

Для достижения поставленных целей планируется использовать язык программирования C# в сочетании с технологией Windows Forms (WinForms) для создания графических пользовательских интерфейсов (GUI). В дополнение к этому будет применяться MetroFramework, который позволит создать современный и стильный интерфейс, соответствующий современным тенденциям в дизайне. Также будут использованы современные инструменты проектирования баз данных, такие как Microsoft SQL Server, а также технологии ADO.NET и Entity Framework для создания пользовательского интерфейса и обработки данных. В результате реализации проекта будет

создана система, позволяющая пользователям добавлять, редактировать и удалять записи о хоккеистах, а также получать статистическую информацию о команде. Использование MetroFramework обеспечит более привлекательный и интуитивно понятный интерфейс, улучшая взаимодействие пользователей с системой. Результаты работы будут иметь практическое применение в сфере спортивного менеджмента, помогая тренерам и менеджерам по работе с командой более эффективно управлять составом. В частности, система предоставит возможность отслеживать количество хоккеистов у каждого тренера, зарплату игроков и среднюю зарплату команды, что поможет в планировании и оптимизации бюджета.

Актуальность данной задачи обусловлена необходимостью повышения эффективности работы спортивных клубов, что может привести к улучшению спортивных результатов и росту популярности команды. Практическая значимость решения заключается в создании удобного инструмента для управления составом команды, который будет способствовать улучшению качества управления и упрощению взаимодействия между всеми участниками процесса.

Нормативные ссылки

В данном курсовом проекте использованы ссылки на следующие нормативные документы:

– ГОСТ Р 7.0.5-2008 СИБИД. Библиографическая ссылка. Общие требования и правила составления;

– ГОСТ 7.12-93 СИБИД. Библиографическая запись. Сокращения слов на русском языке. Общие требования и правила;

– ГОСТ 7.9-95 СИБИД. Реферат и аннотация. Общие требования.

– ГОСТ Р 2.105-2019 ЕСКД. Общие требования к текстовым документам.

– ГОСТ 8.417-2002 ГСИ. Единицы Вычислений.

1. Жизненный цикл программного продукта

1.1 Анализ задачи

Проект «Разработка программного обеспечения о сведениях о хоккеистах-нападающих» направлен на создание эффективной и удобной платформы для управления информацией о хоккеистах. Основной целью проекта является автоматизация процессов обработки данных о хоккеистах, улучшение взаимодействия между тренерами и управленческим персоналом, а также предоставление пользователям актуальной информации о составе команды.

Проектные ограничения:

1. Временные ограничения: Проект должен быть завершен в течение 4 месяцев. Все ключевые этапы, такие как анализ требований, проектирование, разработка и тестирование, должны быть выполнены в строго установленные сроки.
2. Бюджетные ограничения: Необходимо придерживаться заранее установленного бюджета, который должен покрывать все расходы на разработку, тестирование и внедрение программного обеспечения.
3. Технические ограничения: Выбор технологий должен соответствовать навыкам и опыту команды разработчиков, что обеспечит эффективность и качество выполнения проекта.
4. Человеческие ресурсы: В проекте будет ограниченное количество участников команды, что может оказать влияние на скорость разработки и тестирования.

Предлагаемый бюджет проекта:

Предлагаемый объём бюджета составляет около $10,000 - $15,000. Из этой суммы на заработную плату разработчиков планируется выделить $6,000 - $8,000. Лицензии на программное обеспечение займут около $1,000 - $2,000. Для хостинга и серверных ресурсов предусмотрено $2,000 - $3,000. На непредвиденные расходы выделяется $1,000.

Рекомендованный состав команды для реализации проекта:

Для успешной реализации проекта рекомендуется сформировать команду из двух разработчиков программного обеспечения, дизайнера интерфейса, системного аналитика, специалиста по безопасности данных и тестировщика.

Программное обеспечение для разработки ПО:

Язык программирования C#, система управления базами данных MySQL, WinForms, MetroFramework, NuGet Package Manager, Entity Framework.

Последовательность выполнения проекта:

a) Составления плана разработки проекта;

b) Создание базы данных для хранения сведений;

c) Разработка кода ПО;

d) Разработка дизайна интерфейса;

e) Внедрение пользовательского интерфейса;

f) Тестирование работоспособности полученного ПО;

g) Документирование и введение ПО в эксплуатацию.

Программно-аппаратные средства, используемые при выполнении работы:

- языки программирования: C#;
- система управления базами данных: MySQL;
- среда разработки: Visual Studio;
- фреймворки: Entity Framework, MetroFramework;
- ОС: Windows.

Описание назначения ПО:

Данное программное обеспечение предназначено для управления сведениями о хоккеистах-нападающих. Оно предоставляет пользователям удобный интерфейс для работы с данными о хоккеистах, позволяя эффективно отслеживать, редактировать и анализировать информацию. Основные функции и возможности ПО включают:

1. Управление данными о хоккеистах: Создание таблицы с ключевыми сведениями о каждом хоккеисте, включая фамилию хоккеиста, номер игрока, амплуа, дату рождения, дату подписания контракта, фамилию тренера, зарплату и продолжительность контракта.

2. Добавление, редактирование и удаление записей: Пользователи могут легко добавлять новых хоккеистов, изменять существующие записи или удалять ненужные данные.

3. Анализ данных: Программное обеспечение позволяет определять количество хоккеистов, тренирующихся у тренера с заданной фамилией. Возможность нахождения зарплаты наиболее высокооплачиваемого хоккеиста и расчета средней зарплаты всех хоккеистов позволяет анализировать финансовые показатели клуба.

4. Сортировка данных: Пользователи могут сортировать данные в таблице по любому заданному столбцу.

5. Массовое изменение данных: Функция изменения сведений о хоккеистах, подписавших контракт позже заданной даты.

6. Удаление информации о хоккеистах: Возможность удаления информации о хоккеисте с заданной фамилией.

1.2 Разработка технического задания на проектируемую систему

Общие сведения

1.1. Наименование системы

Программное средство «HockeyManager».

2.1. Назначение и цели создания системы

Программное средство «HockeyManager» предназначено для хранения сведений о хоккеистах-нападающих.

Характеристика объектов автоматизации

3.1. Краткое описание работы системы

Краткое описание программного средства «HockeyManager». К

основным направлениям относятся:

- управление данными о хоккеистах, а также их добавление, редактирование и удаление;
- расчёт количества хоккеистов, тренирующихся у тренера с заданной фамилией, а также нахождения зарплаты наиболее высокооплачиваемого хоккеиста и расчёта средней зарплаты всех хоккеистов;
- сортировка данных по любому заданному столбцу.

Требования к информационной системе

4.1. Базовые принципы разработки подсистем

Система должна удовлетворять следующим требованиям:

Пользовательский интерфейс системы должен быть разработан с учетом одной роли пользователя --- Администратора.

Система должна содержать:

- фамилия хоккеиста;
- номер игрока;
- амплуа;
- дата рождения;
- дата подписания контракта;
- фамилия тренера;
- зарплата;
- продолжительность контракта;
- рассчитанное количество хоккеистов;
- нахождение зарплаты наиболее высокооплачиваемого хоккеиста;
- рассчитанная средняя зарплата всех хоккеистов;
- сортировка данных.

4.2. Требования к архитектуре системы

Архитектура программного средства «HockeyManager» является однозвенной.

4.3. Требования к пользователям

Администратор имеет неограниченный доступ к информации программного средства, может добавлять, изменять, удалять информацию.

4.4. Требования к программному обеспечению

Программное обеспечение должно удовлетворять следующим требованиям:

- оперативная память: 4 ГБ/8 Гб;
- жесткий диск: 512/1024 Гб;
- видеокарта: NVIDIA GeForce RTX 4050 Laptop GPU;
- ОС: Windows 10/11.

1.3 Управление проектом

Планирование, поиск рисков разработки

Таблица 1 – Этапы разработки проекта

Номер этапа	Название этапа	Зависимые этапы	Длительность
T1	Сбор требований		8 дней
T2	Анализ требований	T1	8 дней
T3	Проектирование архитектуры	T2	5 дней
T4	Разработка дизайна интерфейса	T3	7 дней
T5	Разработка функционала ПО	T4	14 дней
T6	Интеграция и настройка БД	T4	14 дней
T7	Функциональное тестирование	T5,T6	7 дней
T8	Нагрузочное тестирование	T7	7 дней
T9	Исправление ошибок и доработка	T7	7 дней
T10	Подготовка среды для внедрения	T8,T9	7 дней
T11	Обучение пользователей и	T10	

	настройка		

Таблица 2 – Временной диапазон выполнения работы

Номер этапа	Название этапа	Дата начала этапа	Дата завершения этапа
T1	Сбор требований	03.09.2024	11.09.2024
T2	Анализ требований	11.09.2024	19.09.2024
T3	Проектирование архитектуры	19.09.2024	24.09.2024
T4	Разработка дизайна интерфейса	24.09.2024	01.10.2024
T5	Разработка функционала ПО	01.10.2024	15.10.2024
T6	Интеграция и настройка БД	01.10.2024	15.10.2024
T7	Функциональное тестирование	15.10.2024	22.10.2024
T8	Нагрузочное тестирование	22.10.2024	29.10.2024
T9	Исправление ошибок и доработка	22.10.2024	29.10.2024
T10	Подготовка среды для внедрения	29.10.2024	05.11.2024
T11	Обучение пользователей и настройка	05.11.2024	12.11.2024

Сетевая диаграмма этапов разработки — это графическое представление последовательности и взаимосвязей между различными задачами и этапами проекта (см. рис. 1). Она отображает, как задачи связаны друг с другом, какие из них являются зависимыми, и помогает визуализировать сроки выполнения работ, ресурсы и критический путь проекта.

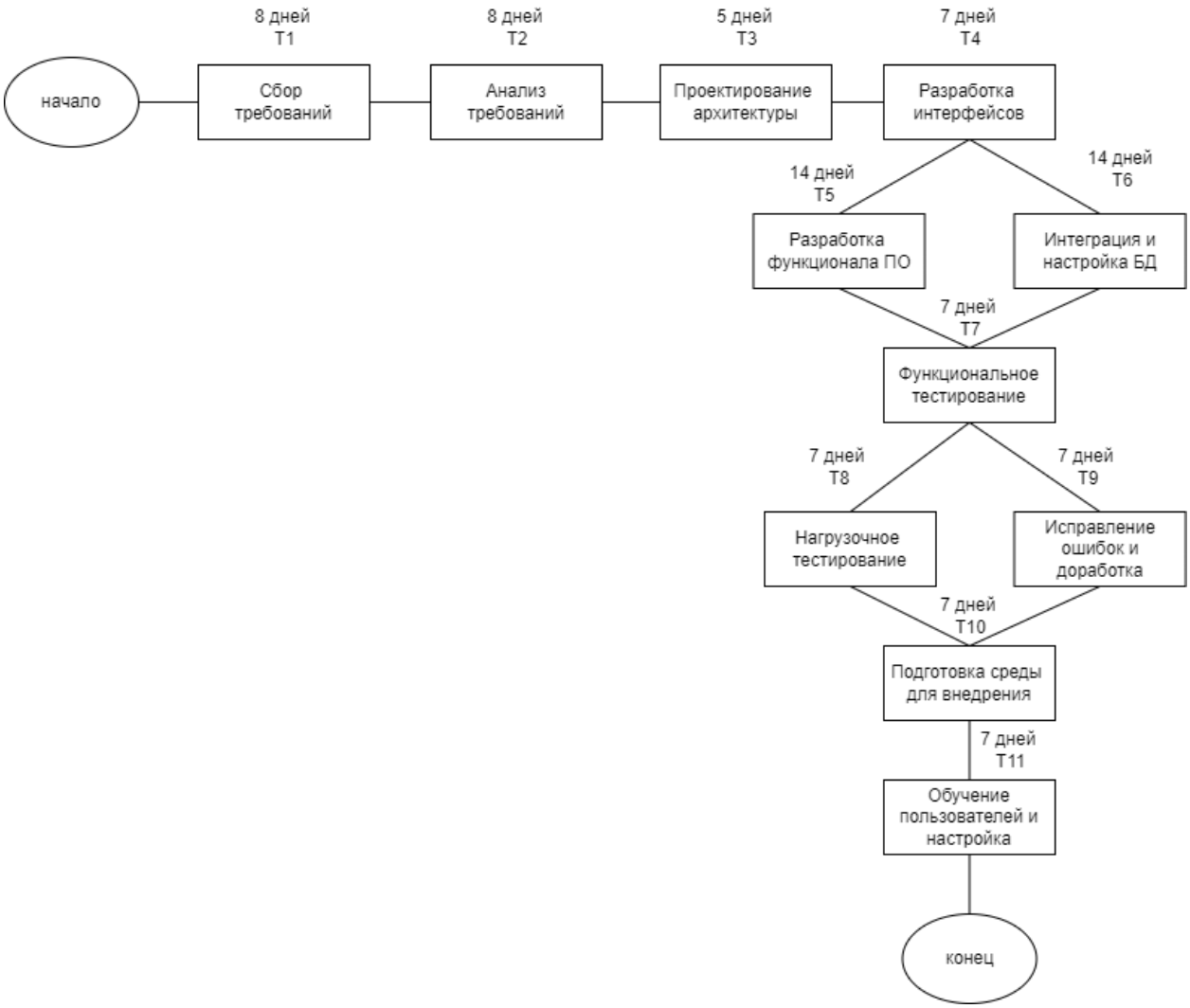


Рисунок 1 – Сетевая диаграмма этапов разработки

Временная диаграмма длительности этапов (диаграмма Ганта) — это графическое представление плана проекта, которое показывает продолжительность различных задач и этапов в виде горизонтальных полос на временной шкале (см. рис. 2).

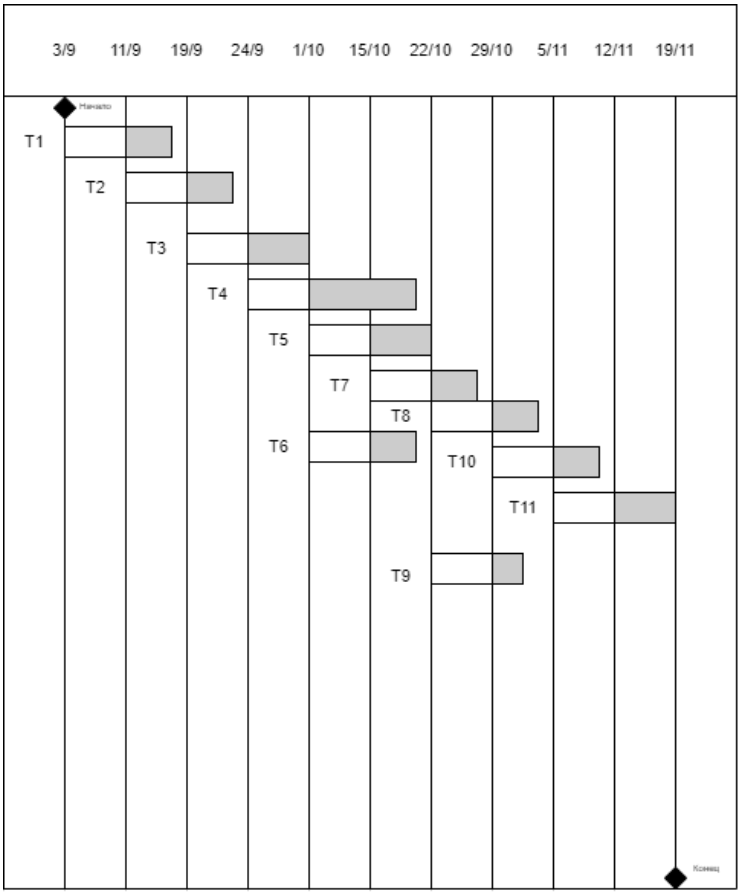


Рисунок 2 – Временная диаграмма длительности этапов

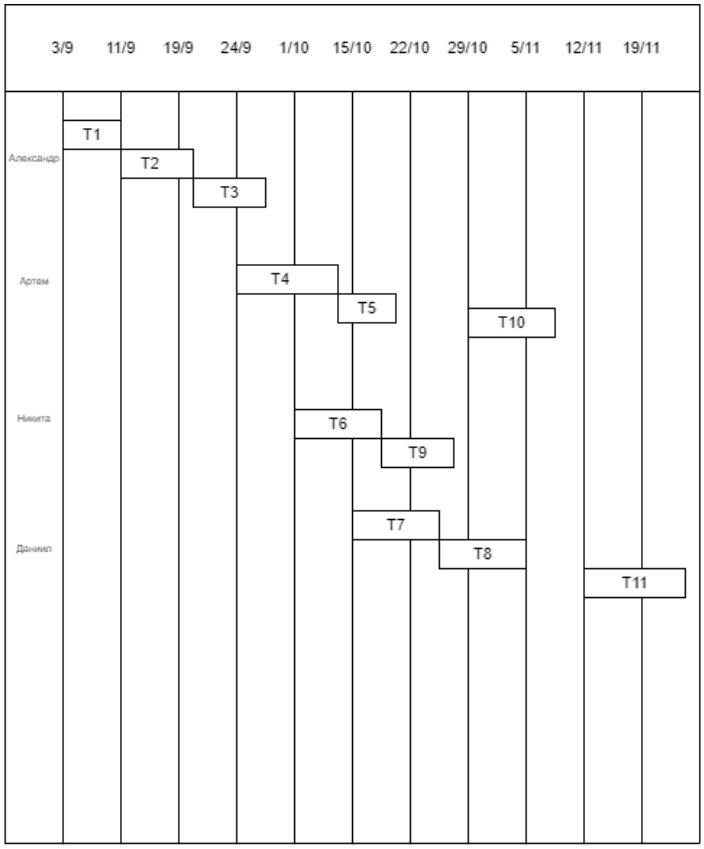


Рисунок 3 – Временная диаграмма распределения работников по этапам

Таблица 3 – Риски при непосредственной разработке ПО

Риск	Название	Описание	Тип
1	Риск устаревания технологий	Используемые технологии могут устареть до завершения проекта, что приведет к необходимости их замены или обновления	Технологический риск
2	Непредвиденные проблемы с интеграцией	Возможны сложности при интеграции различных систем и компонентов, которые могут привести к задержкам или дополнительным затратам.	Технологический риск
3	Недостаток поддержки со стороны руководства	Отсутствие необходимой поддержки и ресурсов от руководства может затруднить выполнение проекта.	Организационный риск
4	Ограниченные возможности ПО для разработки	Используемое ПО может не поддерживать необходимые функции или интеграции, что может потребовать дополнительных средств или изменений в проекте.	Инструментальный риск
5	Изменение требований	Частые изменения требований могут привести к переработке и дополнительным затратам.	Риск, связанный с системными требованиями

Анализ рисков

1. Риск устаревания технологий

Вероятность: 3 (средняя)

Степень ущерба: 4 (высокая)

Описание: Используемые технологии могут устареть до завершения проекта, что приведет к необходимости их замены или обновления.

2. Непредвиденные проблемы с интеграцией

Вероятность: 4 (высокая)

Степень ущерба: 4 (высокая)

Описание: Возможны сложности при интеграции различных систем и компонентов, которые могут привести к задержкам или дополнительным затратам.

3. Недостаток поддержки со стороны руководства

Вероятность: 4 (высокая)

Степень ущерба: 4 (высокая)

Описание: Отсутствие необходимой поддержки и ресурсов от руководства может затруднить выполнение проекта.

4. Ограниченные возможности ПО для разработки

Вероятность: 4 (высокая)

Степень ущерба: 3 (средняя)

Описание: Используемое ПО может не поддерживать необходимые функции или интеграции, что может потребовать дополнительных средств или изменений в проекте.

5. Изменение требований

Вероятность: 4 (высокая)

Степень ущерба: 5 (очень высокая)

Описание: Частые изменения требований могут привести к переработке и дополнительным затратам.

Стратегия планирования рисков

1. Идентификация рисков

- сбор информации: Провести мозговой штурм с командой проекта для выявления потенциальных рисков. Использовать опросы, интервью и исторические данные по аналогичным проектам;
- анализ документов: Изучить проектные документы, спецификации, планы и отчеты для выявления возможных рисков;
- консультации с экспертами: Привлечь внешних экспертов или специалистов по управлению рисками для дополнительной оценки.

Цель: Определить все возможные риски, которые могут повлиять на проект, их причины и последствия.

2. Оценка рисков

- оценка вероятности: Определить вероятность возникновения каждого риска, используя шкалу от 1 до 5, где 1 — очень низкая вероятность, а 5 — очень высокая;
- оценка воздействия: Оценить степень ущерба от каждого риска, используя шкалу от 1 до 5, где 1 — незначительное воздействие, а 5 — катастрофическое;
- приоритизация рисков: Умножить вероятность на степень ущерба, чтобы получить приоритет риска. Это поможет определить, на какие риски следует обратить внимание в первую очередь.

Цель: Оценить и ранжировать риски по их значимости для проектного процесса.

3. Разработка стратегий по смягчению рисков

Разработка планов смягчения, который должен включать:

- избежание: Изменить план проекта, чтобы избежать риска;
- уменьшение: Принять меры для снижения вероятности или воздействия риска;
- перенос: Передать риск третьей стороне (например, через страхование или аутсорсинг);
- принятие: Признать риск и подготовиться к его последствиям, если он возникнет.

Цель: Сформировать конкретные действия и меры для управления каждым риском.

4. Мониторинг и контроль рисков

- регулярные проверки: Внедрить регулярные проверки состояния рисков на каждом этапе проекта. Обновлять списки рисков и планы смягчения по мере необходимости;

− обратная связь: Получать обратную связь от команды и заинтересованных сторон о том, как управляются риски, и вносить коррективы в планы по смягчению;

− отчеты: Включить управление рисками в регулярные отчеты о ходе проекта. Обновлять статус рисков и меры по их управлению.

Цель: Обеспечить актуальность информации о рисках и оперативно реагировать на возникающие проблемы.

5. Документирование и коммуникация

− документирование рисков: Вести подробные записи о каждом риске, планах смягчения, назначенных ответственных лицах и мерах по мониторингу;

− коммуникация: Регулярно информировать команду проекта и заинтересованные стороны о состоянии рисков и мерах по их управлению.

Цель: Обеспечить прозрачность управления рисками и поддерживать осведомленность всех участников проекта.

6. Обучение и развитие

− обучение команды: Организовать обучение для членов команды по вопросам управления рисками, включая методы идентификации, оценки и смягчения рисков;

− развитие навыков: Регулярно обновлять знания команды о лучших практиках и новых инструментах управления рисками.

Цель: Поддерживать команду в актуальном состоянии знаний и навыков по управлению рисками.

2 Модульное проектирование и разработка

2.1 Обоснование выбора среды реализации

Основным средством проектирования приложения выбрана интегрированная среда разработки (IDE) Visual Studio Community Edition. Причины выбора этого ПО достаточно просты: оно удобно в использовании, предоставляет мощные инструменты для разработки, включает предустановленные необходимые библиотеки и пакеты, а также поддерживает интерпретатор языка C#.

В процессе разработки приложения также будет использоваться MetroFramework, который представляет собой библиотеку для создания пользовательских интерфейсов в приложениях Windows Forms (WinForms). MetroFramework позволяет разработчикам создавать современные и стильные интерфейсы, вдохновленные дизайном Metro UI от Microsoft.

Также, в процессе разработки будет использоваться инструмент управления баз данных — SQL Server Object Explorer. Этот инструмент позволяет вам подключаться к базам данных, просматривать их структуру, выполнять запросы и управлять объектами базы данных, такими как таблицы, представления и процедуры.

В процессе разработки будет использоваться Entity Framework — мощный инструмент для работы с базами данных в .NET-приложениях. Entity Framework позволяет разработчикам взаимодействовать с базами данных, используя объектно-ориентированный подход, что значительно упрощает работу с данными.

Для реализации программного кода использовались язык программирования C# и язык работы с базами данных SQL.

C# — это современный, объектно-ориентированный язык программирования, который используется для практически любых целей, начиная разработкой ПО для Windows, сайтов, видеоигр на Unity и заканчивая машинными, глубоким обучением и аналитикой. В данном

проекте он используется для непосредственного написания приложения, первичной обработки данных и взаимодействия приложения с базой данных.

Удобный синтаксис языка программирования, наличие всех необходимых библиотек и баз данных делают процесс разработки требуемого приложения простым и быстрым.

SQL — это декларативный язык программирования, который используется для проектирования баз данных и выполнения различных операций с ними, таких как создание, редактирование, удаление и поиск данных.

Преимущества SQL включают его простоту в использовании, что позволяет даже новичкам быстро осваивать основные команды. Он обеспечивает мощные возможности для манипуляции данными и работы с большими объемами информации. Кроме того, SQL поддерживает стандарты, что делает его совместимым с различными системами управления базами данных. Также благодаря своей структуре он позволяет легко выполнять сложные запросы и анализировать данные, что значительно упрощает процесс принятия решений на основе информации.

2.2 Построение контекстной диаграммы в нотации IDEF0

Контекстная диаграмма — это визуальное представление системы, показывающее её границы и взаимодействия с внешними участниками.

В рамках описания контекста моделирования будут использоваться следующие понятия:

Вход представляет собой информацию или материалы, которые используются или изменяются в процессе работы. Выход, в свою очередь, включает в себя информацию или материалы, которые создаются в результате этой работы. Управление охватывает процедуры, правила, стратегии и стандарты, которые определяют направление и порядок выполнения работы. Механизмы же — это ресурсы, такие как сотрудники,

оборудование и устройства, которые непосредственно осуществляют выполнение задач.

Входные стрелки:

А) Данные о хоккеистах — это информация о новых игроках, их персональных данных, спортивных показателях и контрактных обязательствах, которые поступают в систему для обработки и учета.

Б) Тренерские методики и программы — это разработанные тренерским штабом программы тренировок, тактические схемы и методики подготовки, которые используются для совершенствования навыков хоккеистов.

Выходные стрелки:

А) Отчеты и аналитика — это сформированные системой отчеты о составе команды, статистике игроков, финансовых показателях и эффективности тренировочного процесса.

Б) Управленческие решения — это рекомендации и аналитические выводы, которые помогают руководству клуба принимать решения по трансферной политике, обновлению состава и оптимизации бюджета.

В) Тренировочные планы — это индивидуальные и групповые планы тренировок, сформированные на основе данных о хоккеистах и их спортивных показателях.

Стрелки управления:

А) Законодательство — нормативно-правовые акты, регулирующие спортивную деятельность, трудовое законодательство и защиту персональных данных.

Б) Регламенты и стандарты клуба — внутренние документы, определяющие политику клуба, стандарты работы с игроками и требования к ведению учета.

В) Формат данных — спецификация, определяющая структуру и способ представления данных о хоккеистах в программном обеспечении.

Г) Техническое задание — документированное описание функциональных и нефункциональных характеристик, которые должно удовлетворять разрабатываемое программное обеспечение.

Стрелки механизмов:

А) Интерфейс скаутинга — пользовательский интерфейс, позволяющий скаутам вносить данные о новых игроках, отслеживать их статистику и формировать рекомендации по пополнению состава.

Б) Система управления тренировочным процессом — специализированное программное обеспечение, предназначенное для планирования и контроля тренировочной деятельности.

В) База данных хоккеистов — организованная структура, предназначенная для хранения, управления и обработки данных о хоккеистах, тренерах и контрактах.

Г) Система для обработки и анализа данных — это наше ПО «HockeyManager», предназначенное для упрощения обработки и анализа данных, связанных с хоккеистами. С помощью HockeyManager пользователи могут легко управлять информацией о игроках и тренерах, что значительно повышает эффективность работы спортивного клуба.

Согласно контексту моделирования, построим диаграмму с помощью программного обеспечения «AllFusion Process Modeler». В итоге получим следующий результат (см. рис. 4):

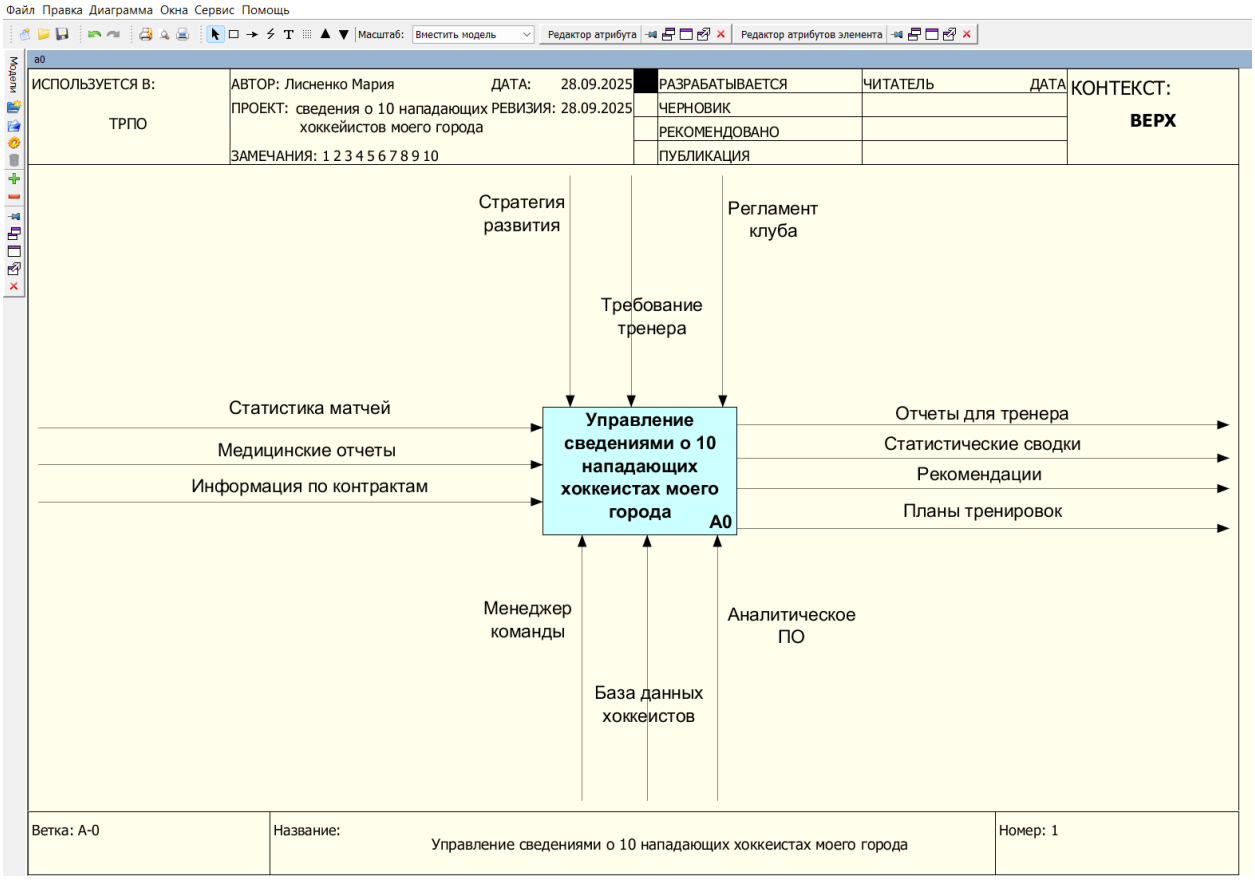


Рисунок 4 – Контекстная диаграмма

2.3 Построение диаграммы декомпозиции в нотации IDEF0

Диаграмма декомпозиции второго уровня, или диаграмма разложения (модель разложения), представляет собой схематическое изображение разбиения системы на подсистемы более низкого уровня. Она позволяет более детально описать внутреннюю структуру системы, выделив основные компоненты и их взаимосвязи.

На следующем этапе проанализируем работу программы поглубже и проведём декомпозицию предыдущей диаграммы (см. рис.5):

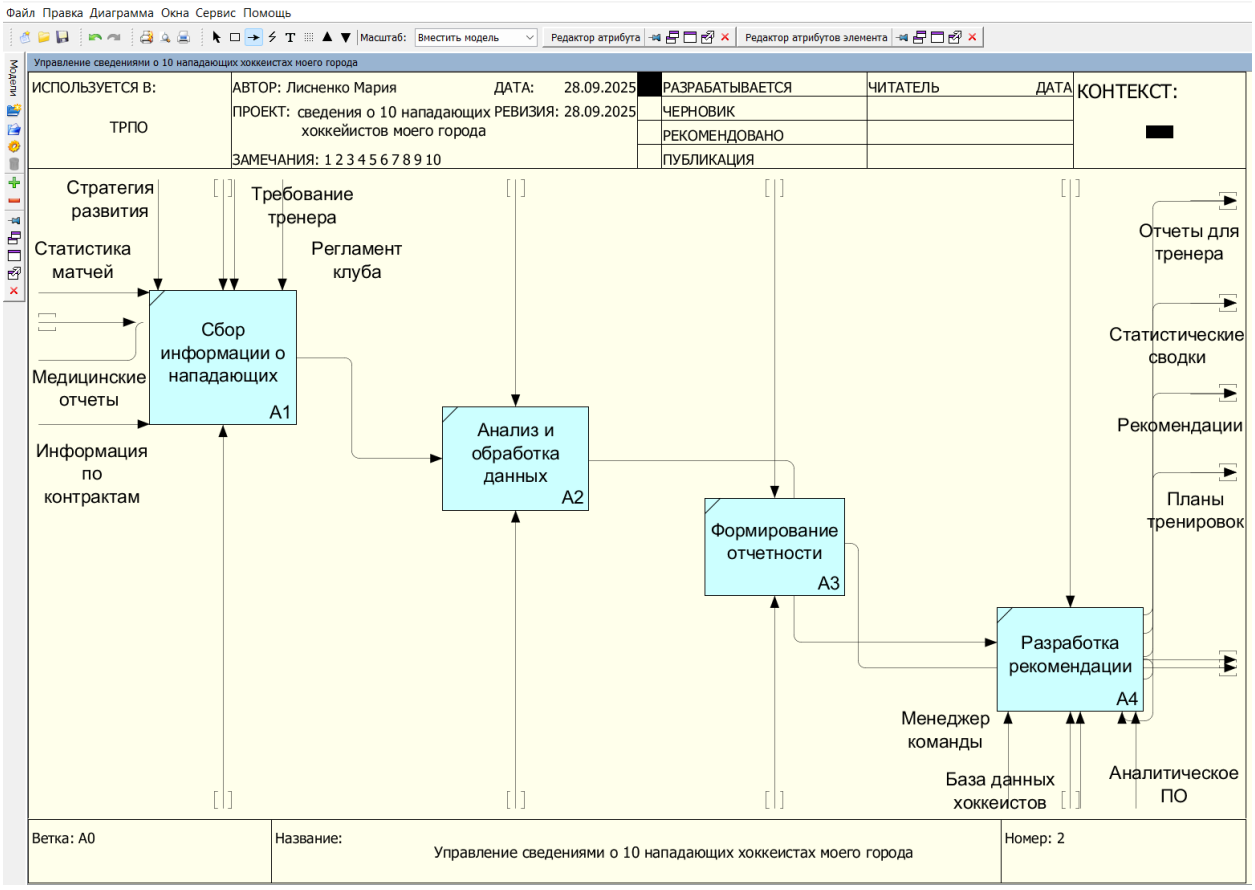


Рисунок 5 – Диаграмма декомпозиции (IDEF0)

Краткое описание каждой работы:

1. Учет и регистрация хоккеистов — включает в себя разработку функционала для внесения данных о новых хоккеистах в систему, создание форм для заполнения персональной информации, спортивных характеристик, контрактных данных. Также включает в себя функционал для прикрепления медицинских документов, сканированных копий контрактов и фотографий игроков. Обеспечивает ведение базы данных с актуальной информацией о всех хоккеистах команды.

2. Анализ и оценка спортивных показателей — разработка функционала для отслеживания и анализа спортивных результатов хоккеистов. Включает в себя создание базы данных с статистикой игр, показателями эффективности, данными о травмах и восстановлении. Разработка интерфейса для тренерского штаба, который позволяет вносить

оценки за тренировки и игры, а также систему автоматического уведомления о падении спортивных показателей или достижении целевых значений.

3. Планирование тренировочного процесса — разработка функционала для создания индивидуальных и групповых тренировочных программ. Включает в себя создание базы данных с упражнениями, тактическими схемами и методиками подготовки. Создание интерфейса для тренеров, где можно формировать расписание тренировок, назначать индивидуальные задания, отслеживать прогресс выполнения и корректировать планы в зависимости от результатов хоккеистов.

4. Мониторинг и отчетность — разработка функционала, позволяющего руководству клуба и тренерскому штабу отслеживать текущее состояние команды и отдельных игроков. Включает в себя создание панели управления с ключевыми показателями, генерацию отчетов по запросу, автоматическое формирование аналитических выкладок по финансовым показателям, спортивным результатам и эффективности тренировочного процесса. Обеспечивает возможность просмотра истории изменений и прогнозирования развития игроков.

Взаимосвязи между процессами:

- данные из блока А1 (Учет хоккеистов) используются в блоках А2 и А3 для анализа и планирования;
- результаты анализа из А2 влияют на планирование тренировок в А3;
- все процессы передают данные для мониторинга и отчетности в блок А4;
- управляющие воздействия (законодательство, регламенты) применяются ко всем процессам системы.

2.4 Определение компонентов, их связывание при построении модели IDEF3

Модель IDEF3 — это методология, используемая для моделирования и описания бизнес-процессов.

Краткое описание каждой работы:

1. Анализ текущего состава команды

Оценка текущего состояния команды, анализ позиций хоккеистов, их спортивной формы и статистики. Определение слабых и сильных позиций в составе для выявления потребности в усилении команды.

2. Скаутинг и поиск новых хоккеистов

Активный поиск и оценка потенциальных кандидатов для пополнения состава. Включает анализ выступлений в других клубах, просмотр матчей, оценку физических кондиций и игровых качеств хоккеистов.

3. Медицинское обследование и тестирование

Проведение комплексного медицинского обследования кандидатов, включая функциональную диагностику, тесты на физическую подготовку и оценку состояния здоровья. Определение готовности хоккеиста к профессиональным нагрузкам.

4. Заключение контрактов и оформление документов

Оформление трудовых отношений с хоккеистами, подготовка и подписание контрактов, согласование условий с агентами. Включает юридическое сопровождение и регистрацию игроков в спортивных федерациях.

5. Интеграция в команду и адаптация

Организация процесса адаптации новых хоккеистов в коллективе. Включает знакомство с командными правилами, тактическими схемами, тренерским штабом и организацию совместных тренировок с командой.

6. Индивидуальная тренировочная программа

Разработка и реализация индивидуального плана тренировок для каждого

хоккеиста. Включает работу над слабыми сторонами, развитие сильных качеств и подготовку к конкретным игровым ситуациям.

7. Участие в командных тренировках и играх

Регулярное участие хоккеиста в общих тренировках, тактических занятиях и официальных матчах. Мониторинг игровой практики и эффективности в реальных match-условиях.

8. Анализ эффективности и корректировка планов

Постоянный мониторинг спортивных показателей, анализ статистики выступлений и внесение корректировок в тренировочный процесс. Формирование отчетов для тренерского штаба и руководства клуба.

Диаграмма декомпозиции в нотации IDEF3 (см. рис.6):

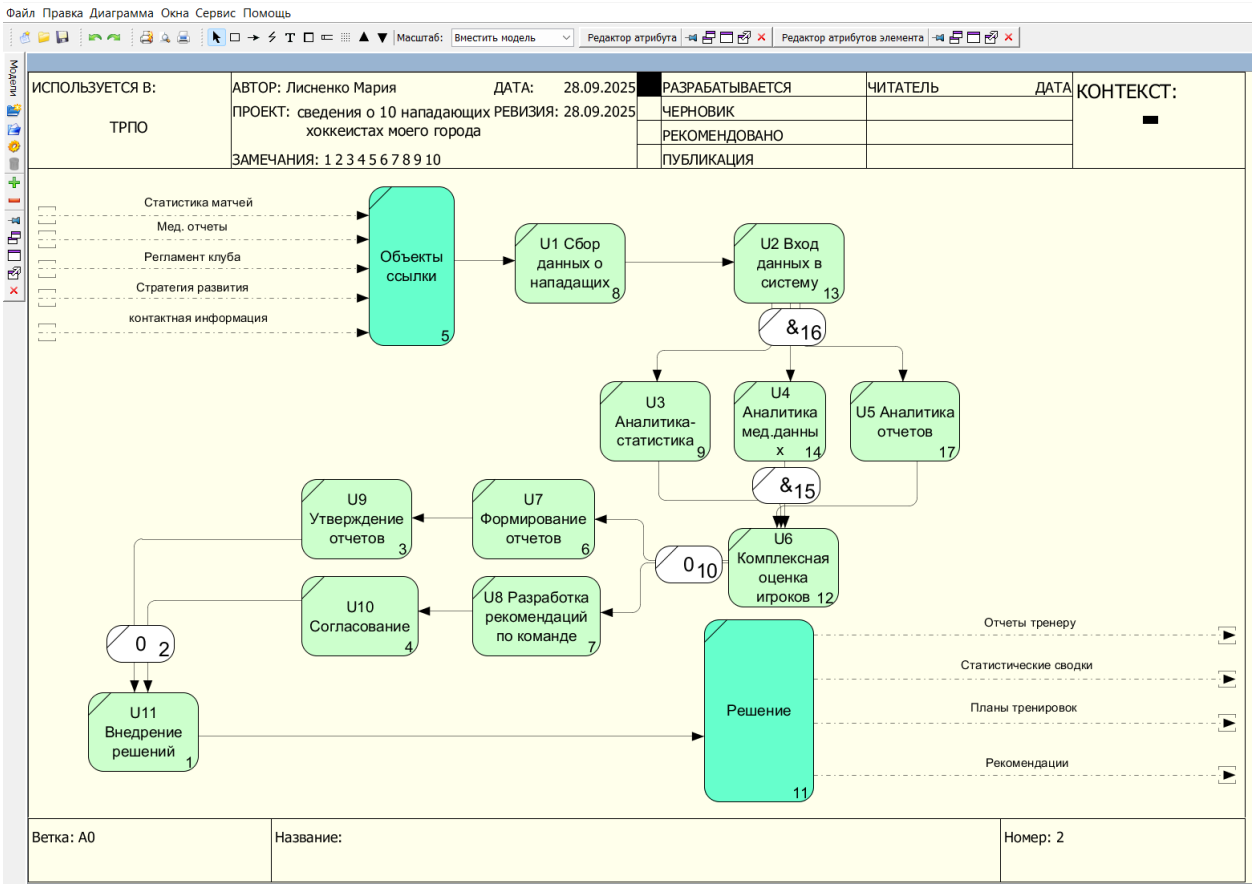


Рисунок 6 – Диаграмма декомпозиции (IDEF3)

2.5 Определение компонентов для построения диаграммы потоков данных (DFD)

DFD — это диаграмма потоков данных, используемая для визуального представления потока информации в системе. Она помогает анализировать, проектировать и документировать системы, показывая, как данные перемещаются между процессами, хранилищами данных и внешними сущностями (см. рис. 7 – 8).

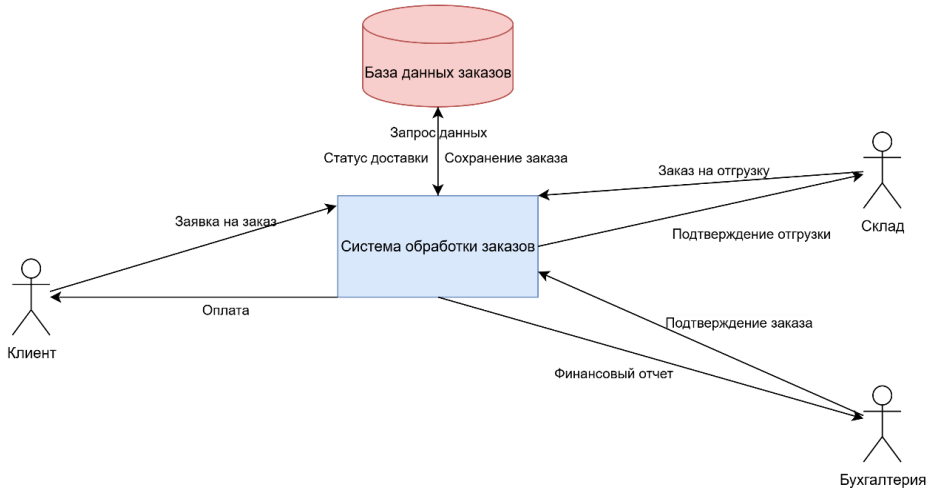


Рисунок 7 DFD – – диаграмма (Система управления заказами)

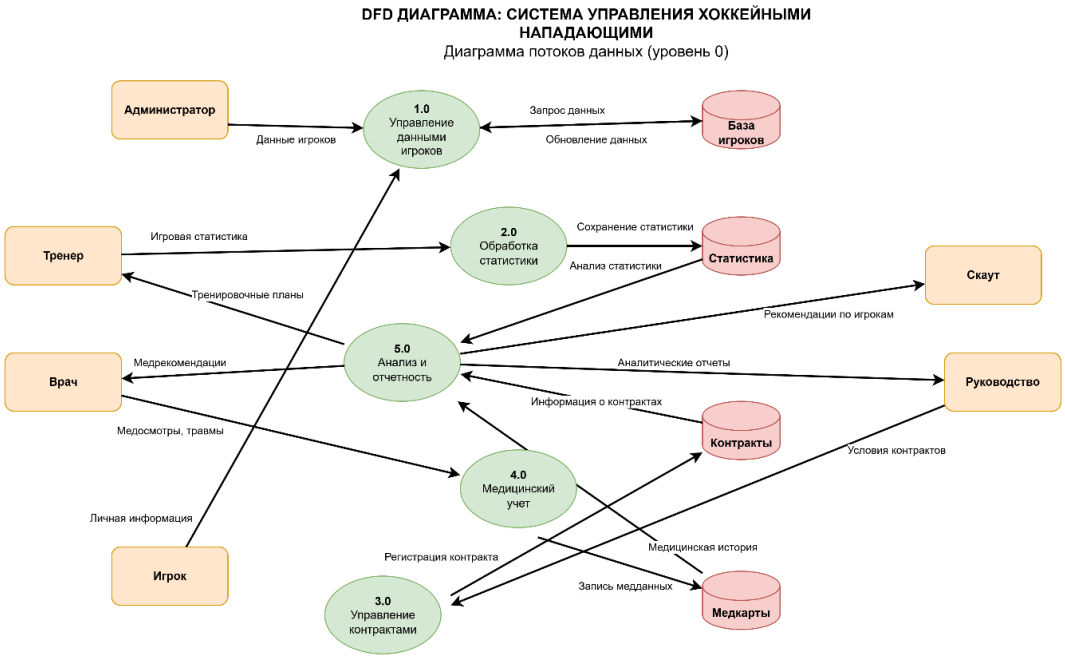


Рисунок 8 – DFD – диаграмма (Обновление статуса товара в приложении)

2.6 Определение компонентов, объектов, действий бизнес-процесса при построении модели BPMN. Определение последовательности действий при построении модели BPMN

BPMN — это стандарт, предназначенный для моделирования бизнес-процессов (см. рис. 9).

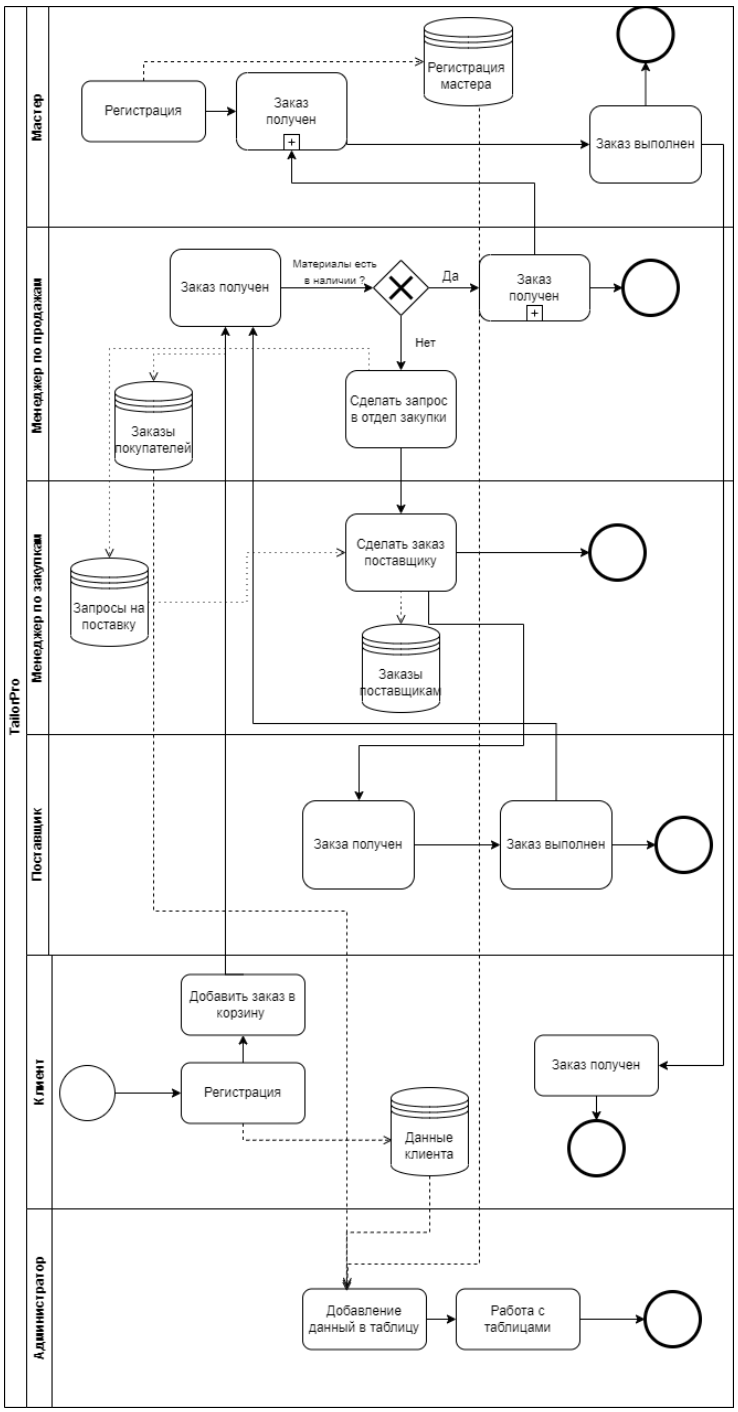


Рисунок 9 – BPMN-диаграмма

3 Структурное проектирование и разработка

3.1 Построение диаграммы состояний

Диаграмма состояния — это графическое представление, описывающее поведение объекта в системе через последовательность состояний и переходов между ними (см. рис. 10).

В рамках диаграммы состояний будут присутствовать такие состояния и переходы, как:

1. Состояния:

a. Кандидат - первичное рассмотрение хоккеиста, сбор базовой информации;

b. На рассмотрении - углубленный анализ спортивных показателей и медицинских данных;

c. На тестировании - хоккеист проходит практические испытания в тренировочном процессе;

d. В команде - хоккеист официально зачислен в состав и участвует в деятельности клуба;

e. В резерве - хоккеист находится в резервном составе, может быть привлечен к основным играм;

f. Завершил карьеру - хоккеист завершил выступления в клубе.

2. Переходы:

a. Подача заявки - информация о хоккеисте поступает в скаутский отдел;

b. Одобрение кандидатуры - хоккеист допускается к углубленному тестированию;

c. Успешное тестирование - хоккеист проходит все испытания и рекомендуется к подписанию контракта;

d. Подписание контракта - хоккеист официально становится игроком клуба;

e. Перевод в резерв - хоккеист перемещается в резервный состав по решению тренерского штаба;

f. Возврат в основу - хоккеист возвращается в основной состав из резерва;

g. Завершение контракта - хоккеист завершает выступления в клубе.

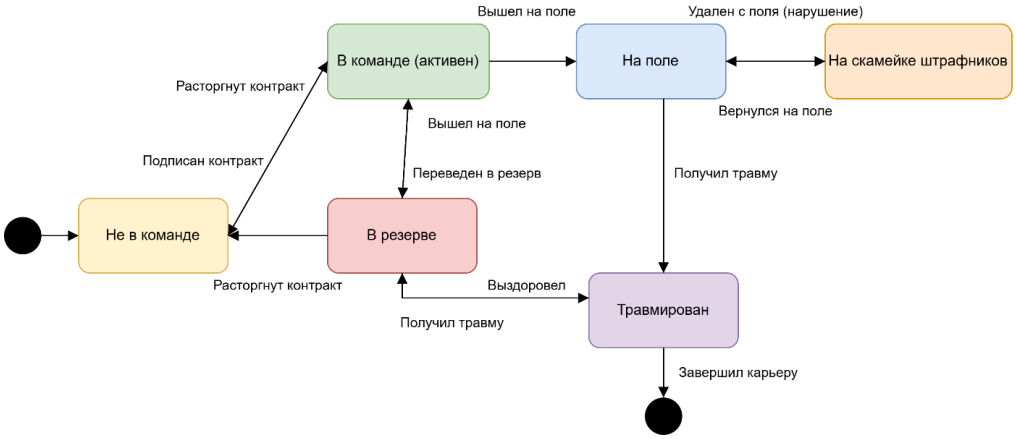


Рисунок 10 – Диаграмма состояний

3.2 Построение диаграммы последовательности. Построение диаграммы кооперации

Диаграмма последовательности — это графическое представление, которое иллюстрирует динамическое взаимодействие между объектами в системе во времени (см. рис. 11).

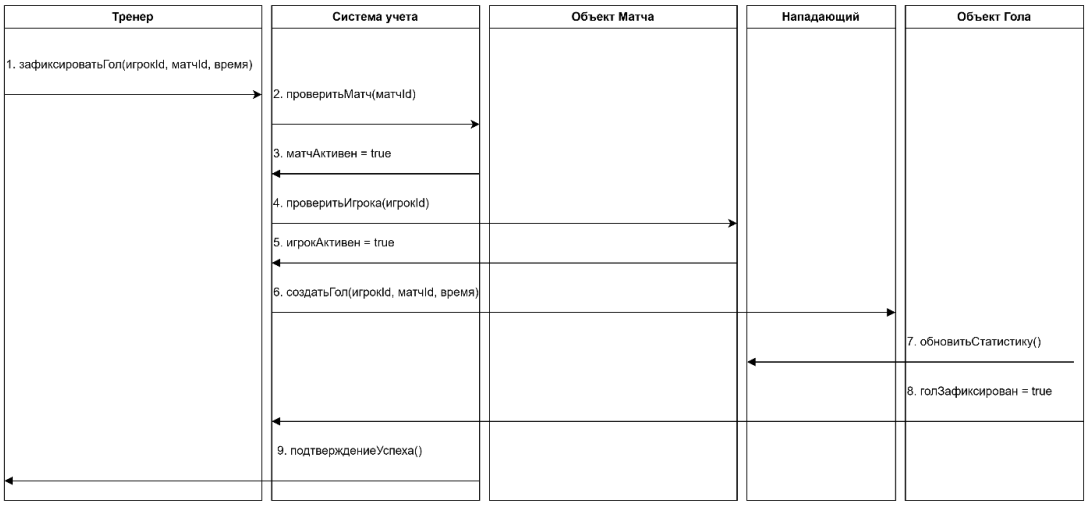


Рисунок 11 – Диаграмма последовательности

Диаграмма кооперации — это графическое представление, показывающее, как объекты системы взаимодействуют друг с другом для выполнения определенной задачи или сценария. Она акцентирует внимание на структуре и взаимосвязях объектов, а также на порядке и типах сообщений, которые они обмениваются (см. рис. 12).

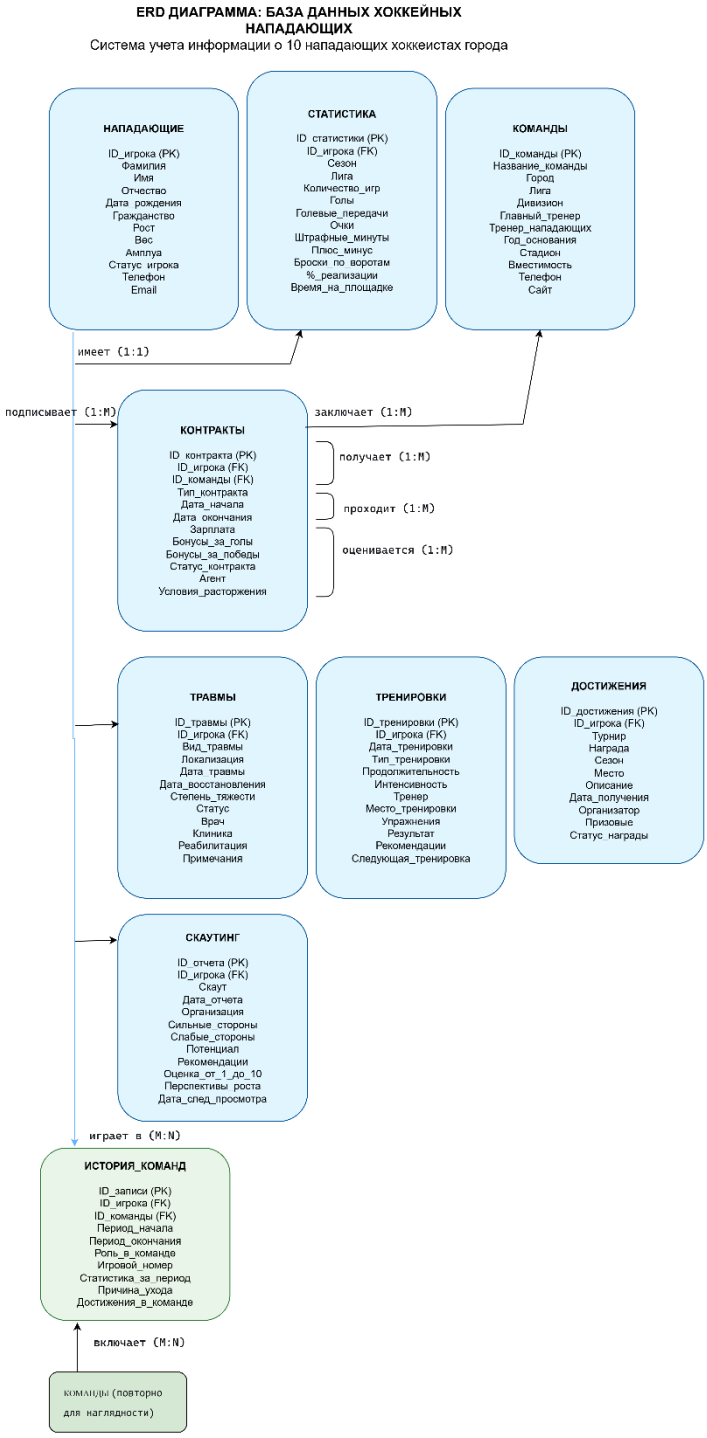


Рисунок 12 – Диаграмма кооперации

3.3 Построение диаграммы компонентов

Диаграмма компонентов — это визуальное представление архитектуры программной системы, которое показывает, как различные программные компоненты взаимодействуют друг с другом и какие интерфейсы они используют для обмена данными (см. рис. 13).

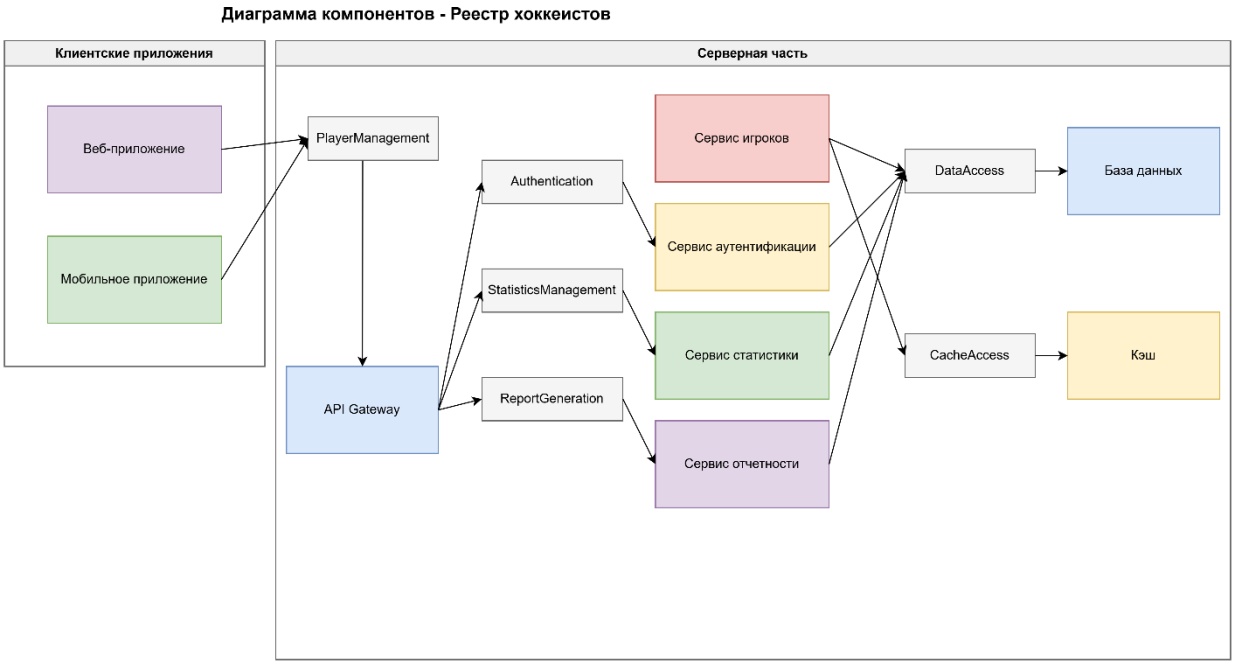


Рисунок 13 – Диаграмма компонентов

4 Объектно-ориентированное проектирование и разработка

4.1 Построение диаграммы Сущность-связь

Сущность-связь диаграмма — это визуальное представление данных и их взаимосвязей в базе данных (см. рис. 14).

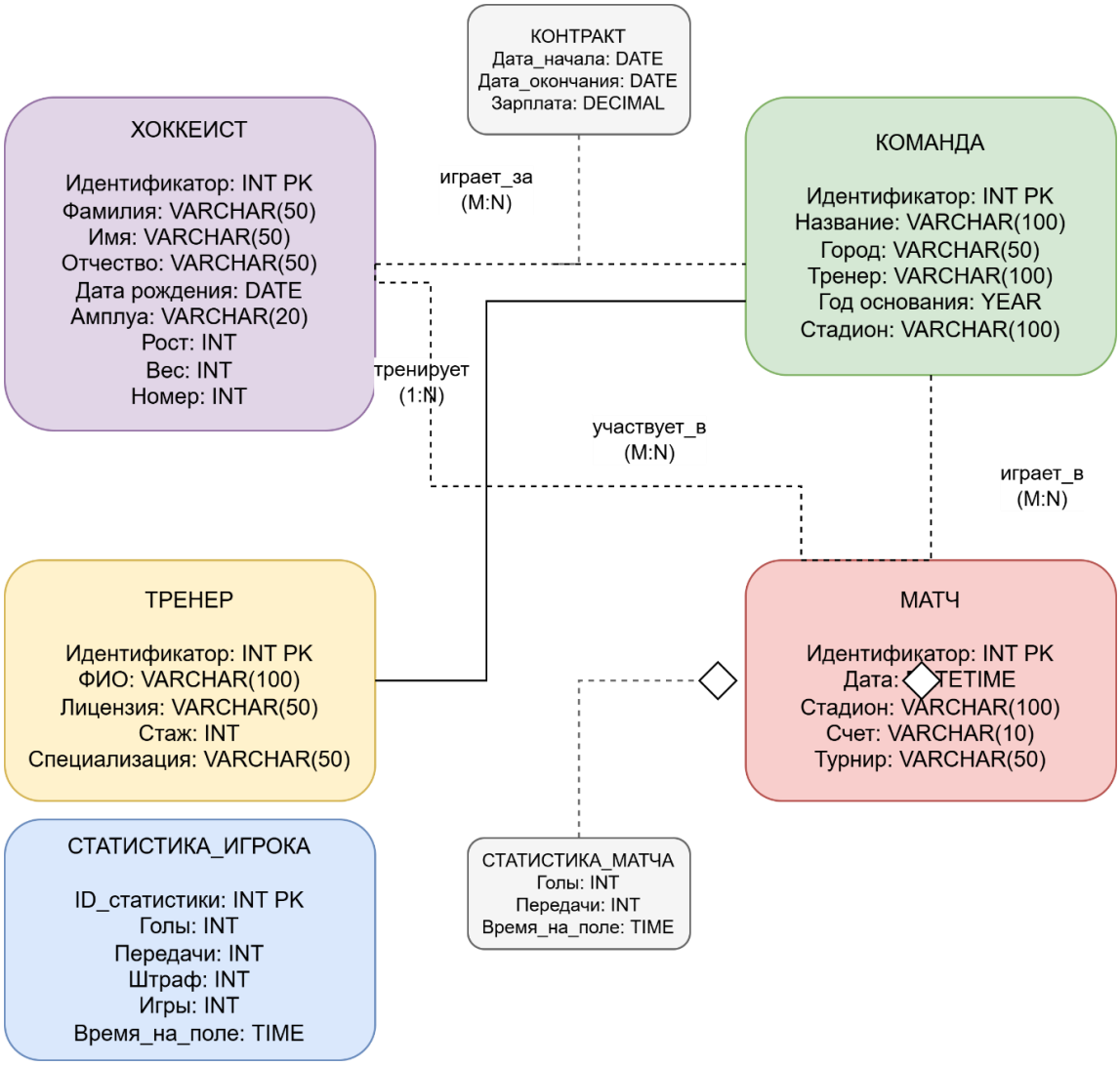


Рисунок 14 – Диаграмма «Сущность-связь»

4.2 Построение диаграммы вариантов использования

Диаграмма вариантов использования — это визуальное представление взаимодействий между пользователями (или другими системами) и системой,

которое помогает понять функциональные требования к системе (см. рис. 15).

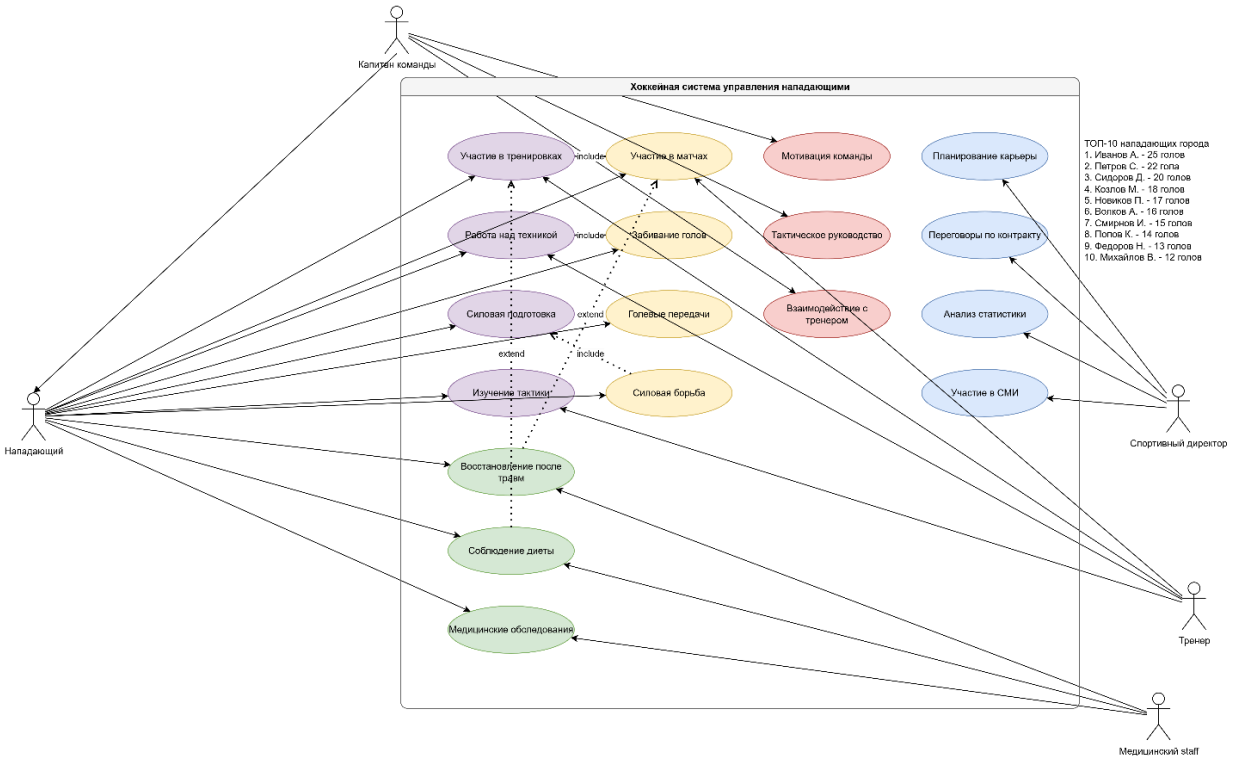


Рисунок 15 – Диаграмма прецедентов

4.3 Построение диаграммы деятельности

Диаграмма деятельности — это визуальное представление последовательности действий, процессов или рабочих потоков в системе, используемое для моделирования бизнес-процессов и алгоритмов (см. рис. 16).

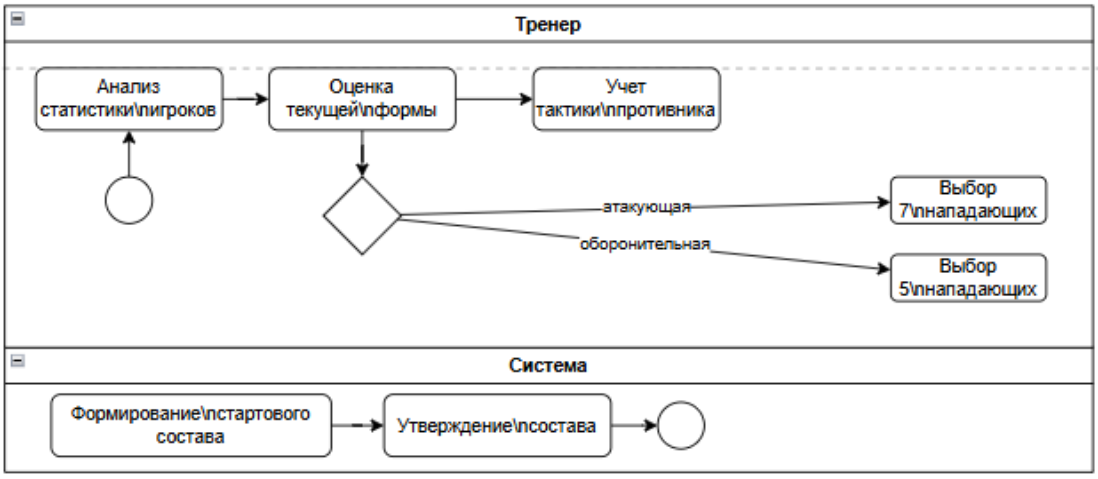


Диаграмма 2: Проведение тренировки

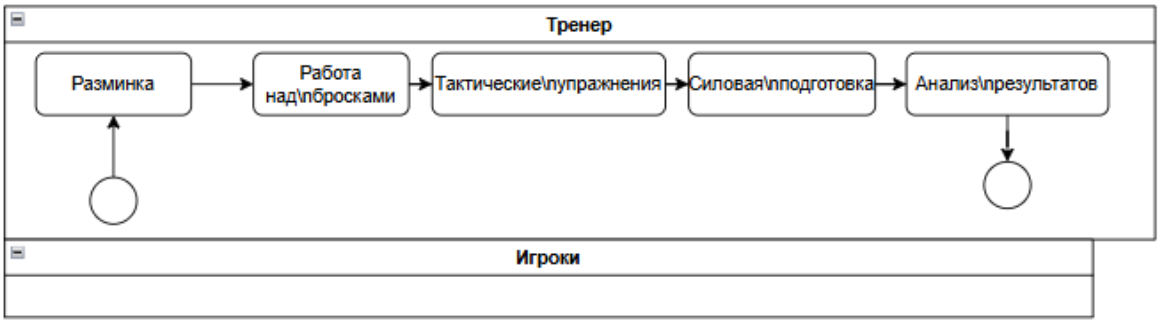


Диаграмма 3: Анализ игровой статистики

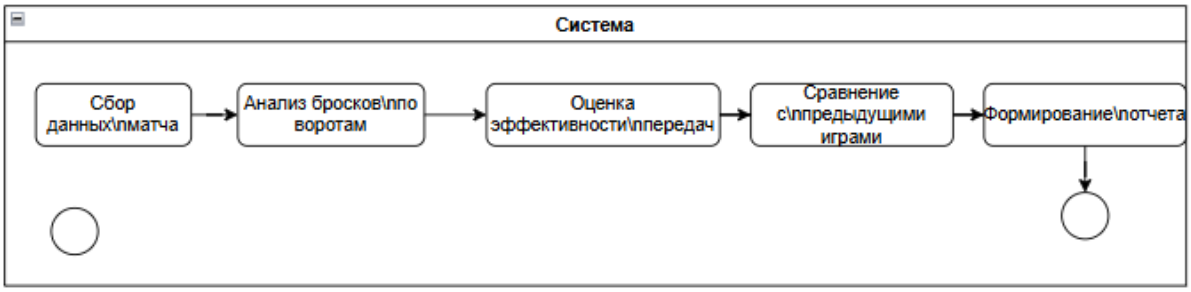


Диаграмма 4: Замена игроков во время матча

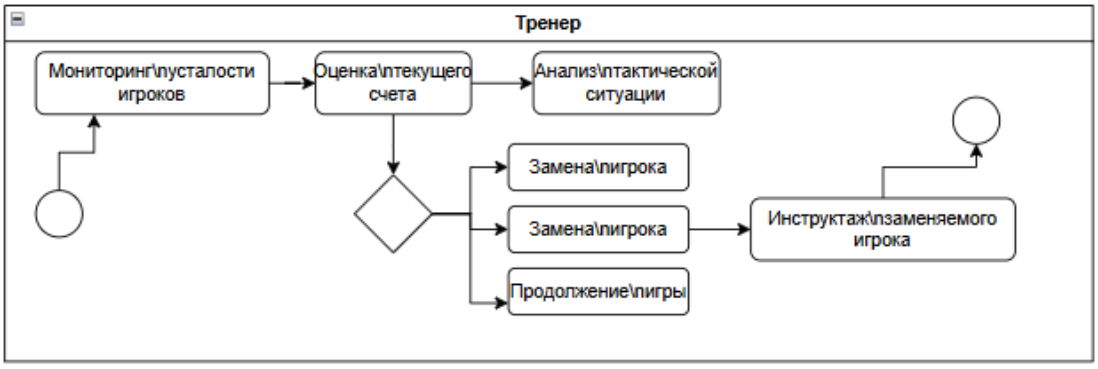
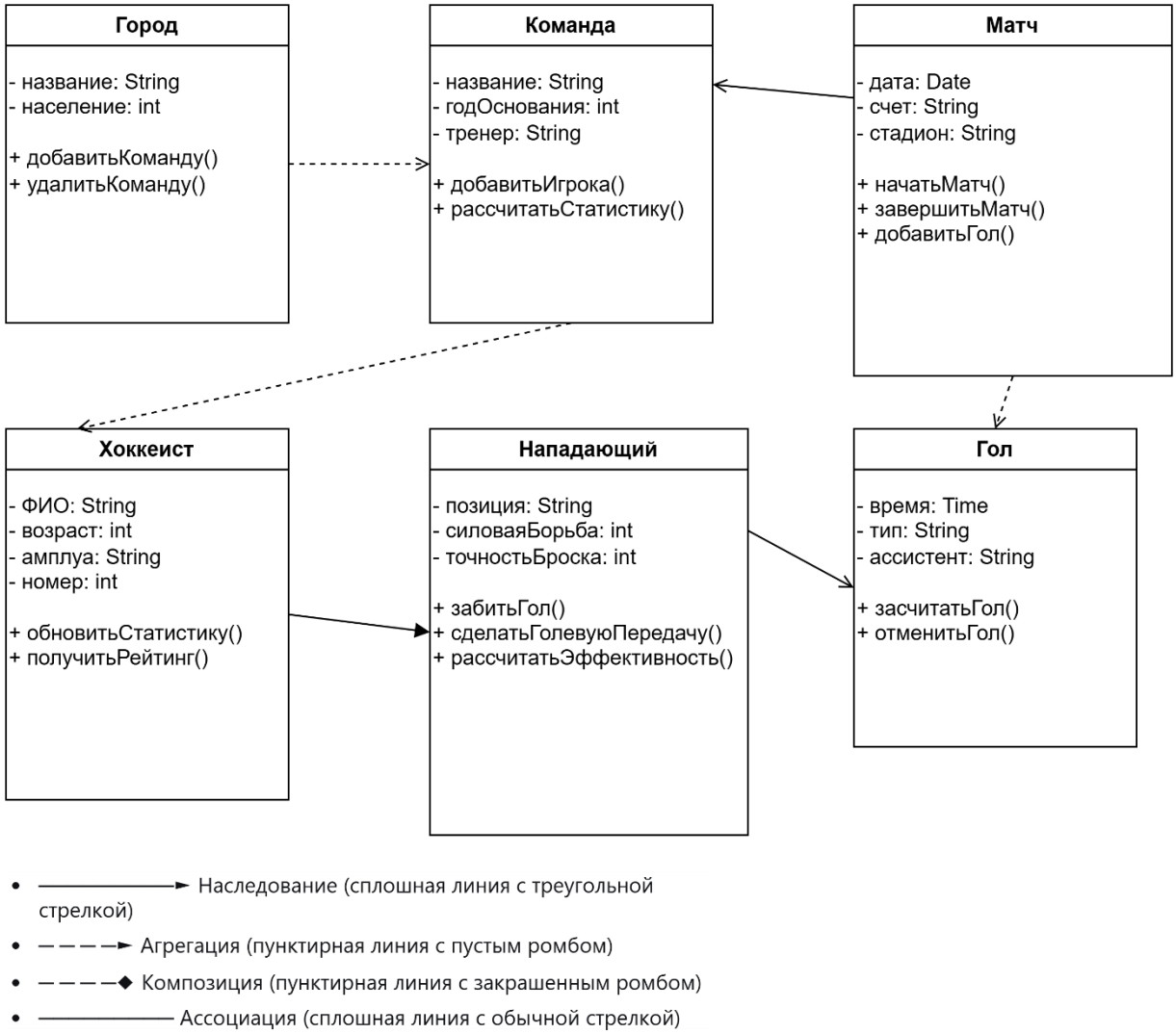


Рисунок 16 – Диаграмма деятельности

4.4 Построение диаграммы классов

Диаграмма классов — это графическое представление классов в системе, их атрибутов, методов и отношений между ними, используемое для моделирования структуры программного обеспечения и понимания его архитектуры (см. рис. 17).



- ————► Наследование (сплошная линия с треугольной стрелкой)
- – – – –► Агрегация (пунктирная линия с пустым ромбом)
- – – – –◆ Композиция (пунктирная линия с закрашенным ромбом)
- ———— Ассоциация (сплошная линия с обычной стрелкой)

Рисунок 17 – Диаграмма классов

5 Инструментальные средства тестирования, проектирования и разработки программного обеспечения.

5.1 Разработка пользовательского интерфейса

При разработки интерфейса пользовательского были изучены профили пользователей и основные сценарии, указанные в таблице 4-6.

Таблица 4 – Основные профили пользователей

Профиль 1: Скаут / Аналитик команды ("Алексей Сергеевич")

Характеристика	Описание
Социальные характеристики	Мужчина, 30-45 лет, высшее образование, русскоязычный, высокий уровень владения ПК
Мотивационно-целевая среда	Прямая производственная необходимость в поиске перспективных игроков. Высокая мотивация к обучению, требует глубокого анализа статистики.
Навыки и умения	Прошел полный тренинг работы с системой. Уверенно работает с таблицами, фильтрами и экспортом данных.
Требования к ПО	Сложная фильтрация и сортировка игроков по множеству параметров. Визуализация статистики (графики, диаграммы). Экспорт данных в Excel. Быстрый отклик системы.
Задачи пользователя	Поиск игроков по заданным критериям (количество голов, передач, игровое амплуа). Сравнительный анализ игроков. Просмотр подробной карточки хоккеиста. Формирование отчетов.

Таблица 5 – Основные профили пользователей

Профиль 2: Тренер ("Дмитрий Петрович")

Характеристика	Описание
Социальные характеристики	Мужчина, 40-60 лет, высшее физкультурное образование, русскоязычный, средний уровень владения ПК.
Мотивационно-целевая среда	Производственная необходимость для формирования состава и тактических схем. Мотивация к обучению средняя, ценит простоту и наглядность.
Требования к ПО	Простой и интуитивно понятный интерфейс. Быстрый доступ к ключевой статистике текущего состава. Возможность просмотра динамики показателей игрока.
Задачи пользователя	Просмотр списка своих хоккеистов. Анализ текущей формы игроков. Отслеживание игровой статистики (голы, передачи, +/-). Формирование заявки на матч.

Описание функциональности приложения

На основе целей и задач пользователей сформирован следующий перечень функций:

- аутентификация и авторизация;
- просмотр списка хоккеистов;
- сложный поиск и фильтрация хоккеистов;
- просмотр подробной карточки хоккеиста;
- добавление нового хоккеиста в реестр;
- редактирование данных хоккеиста;
- ввод и редактирование статистики по матчам;
- визуализация статистики (графики и диаграммы);
- формирование и экспорт отчетов (PDF, Excel);
- управление пользователями (для администратора).

Функциональные блоки, операции и объекты указаны в таблице 6:

Таблица 6 – Основные операции и объекты

Функциональный блок	Операции	Объекты
Управление игроками	Просмотр, добавление, редактирование, удаление, поиск, фильтрация.	Хоккеист, Статистика, Команда
Работа со статистикой	Ввод, просмотр, редактирование, визуализация (построение графиков).	Матч, Гол, Передача, Бросок
Аналитика и отчетность	Формирование отчета, экспорт данных, сравнительный анализ.	Отчет, График, Диаграмма
Администрирование	Управление ролями, просмотр логов, резервное копирование.	Пользователь, Роль, Лог

4. Множество пользовательских сценариев

Сценарий 1: Поиск перспективного нападающего (для Скаута)

1. Алексей Сергеевич открывает систему и входит под своим логином и паролем.
2. На главной странице он переходит в раздел "Игроки".
3. Использует расширенный поиск: устанавливает фильтры Амплуа = Нападающий, Количество голов в сезоне > 15, Возраст < 25.
4. Система отображает список из 5 хоккеистов, соответствующих критериям.
5. Алексей Сергеевич кликает на фамилию первого игрока в списке, чтобы открыть его подробную карточку.
6. В карточке он изучает график динамики голов по месяцам, просматривает видео с голевыми моментами (если есть).

7. Он нажимает кнопку "Сравнить", выбирает еще двух игроков из списка и анализирует их ключевые показатели в сводной таблице.

8. По итогам анализа экспортирует сводную таблицу в Excel.

Сценарий 2: Анализ формы игроков перед матчем (для Тренера)

1. Дмитрий Петрович заходит в систему. Главный экран сразу показывает ему "Панель тренера" с краткой статистикой его основной команды.

2. Он кликает на раздел "Мой состав". Система показывает список его хоккеистов с основными показателями: голы, передачи, полезность (+/-) в последних 5 играх.

3. Он сортирует таблицу по столбцу "Голы" (убывание), чтобы увидеть самых результативных игроков в текущей форме.

4. Он нажимает на фамилию одного из нападающих, который не забивал в последних 3 играх. Открывается карточка игрока, где тренер смотрит график "Бросков в створ ворот" за последний месяц, чтобы понять, падает ли его игровая активность.

5. Убедившись в необходимости дать игроку отдых, тренер возвращается в "Мой состав" и с помощью перетаскивания исключает его из заявки на следующий матч.

Сценарий 3: Ввод статистики после матча (для Аналитика)

1. После завершения матча аналитик заходит в систему и находит матч в разделе "Календарь".

2. Он нажимает кнопку "Ввести статистику" для этого матча.

3. Система загружает форму с двумя составами команд. Аналитик проставляет голы, передачи, штрафные минуты для каждого игрока, выбирая действия из выпадающих списков и указывая время.

4. После сохранения система автоматически пересчитывает общую статистику для каждого игрока в реестре.

5.2. Разработка информационной базы проекта

Прежде чем приступить к созданию физической модели, мы разработаем информационную базу проекта и модель ERD для базы данных:

Таблицы, в которой отображаются все используемые в БД атрибуты и тип этих атрибутов (см. таблица 7-9):

Таблица 7. Чек-лист функционального тестирования

№	Компонент/Функция	Статус	Примечания
1	Аутентификация		
1.1	Вход с корректными данными	☑	
1.2	Вход с некорректным логином	☑	
1.3	Вход с некорректным паролем	☑	
1.4	Выход из системы	☑	
2	Управление игроками		
2.1	Добавление нового игрока	☑	
2.2	Редактирование данных игрока	☑	
2.3	Удаление игрока	☑	
2.4	Поиск игрока по фамилии	☑	
2.5	Фильтрация по позиции	☑	
3	Статистика		
3.1	Ввод статистики за матч	☑	
3.2	Просмотр общей статистики	☑	
3.3	Сравнение игроков	☑	

№	Компонент/Функция	Статус	Примечания
3.4	Графики эффективности	☑	
4	Отчетность		
4.1	Формирование отчета по сезону	☑	
4.2	Экспорт в PDF	☑	
4.3	Экспорт в Excel	☑	
5	Навигация		
5.1	Переход между разделами	☑	
5.2	Возврат на предыдущую страницу	☑	
5.3	Обновление данных	☑	

Таблица 8. Детальные тест-кейсы

ID	Название теста	Предусловия	Шаги выполнения	Ожидаемый результат	Статус
AUTH-01	Успешная аутентификация	Пользователь зарегистрирован в системе	1. Открыть страницу входа 2. Ввести логин: coach_dmitry 3. Ввести пароль: ******** 4. Нажать "Войти"	Происходит переход на главную панель управления	☑ PASS
AUTH-02	Неуспешная аутентифик	-	1. Открыть страницу входа	Отображается сообщени	☑ PASS

ID	Название теста	Предусловия	Шаги выполнения	Ожидаемый результат	Статус
	ация		2. Ввести неверный логин 3. Ввести любой пароль 4. Нажать "Войти"	е "Неверный логин или пароль"	
PLAYER-01	Добавление нового хоккеиста	Пользователь авторизован как тренер	1. Перейти в раздел "Игроки" 2. Нажать "Добавить игрока" 3. Заполнить обязательные поля 4. Нажать "Сохранить"	Игрок появляется в общем списке, присваивается ID	✅ PASS
PLAYER-02	Поиск игрока	В системе есть несколько игроков	1. Перейти в раздел "Игроки" 2. В поле поиска ввести "Овечкин" 3. Нажать Enter	Отображается только игрок с фамилией "Овечкин"	✅ PASS
STAT-01	Ввод статистики матча	Завершен матч, пользователь авторизован	1. Перейти в "Матчи" 2. Выбрать завершенный матч 3. Нажать	Статистика отображается в профилях игроков	✅ PASS

ID	Название теста	Предусловия	Шаги выполнения	Ожидаемый результат	Статус
			"Ввести статистику" 4. Заполнить данные по игрокам 5. Сохранить		
REPORT-01	Формирование отчета	Заполнена статистика за сезон	1. Перейти в "Отчеты" 2. Выбрать период "Сезон 2024" 3. Нажать "Сформировать отчет" 4. Выбрать "Экспорт в PDF"	Создается PDF-файл с полной статистикой	✅ PASS

Результаты тестирования

Таблица 9. Сводка результатов тестирования

Тип тестирования	Всего тестов	Пройдено	Не пройдено	Успешность
Модульное тестирование	45	42	3	93.3%
Интеграционное тестирование	28	26	2	92.9%
Системное тестирование	35	33	2	94.3%

Тип тестирования	Всего тестов	Пройдено	Не пройдено	Успешность
Пользовательское тестирование	15	15	0	100%
Итого	123	116	7	94.3%

5.3. Проведение тестирования программной системы, выявление, анализ и исправление ошибок

Тестирование — это процесс проверки работоспособности программы или ее отдельных модулей.

Цель нашего тестирования — проверить, что функция обновления заказа в базе данных работает правильно.

В рамках нашего проекта мы создадим новый элемент – проект модульного тестирования на платформе .NET Framework. Этот проект позволит нам систематически проверять функциональность нашего приложения, обеспечивая его надежность и стабильность.

Далее откроем вкладку «Обозреватель тестов» и запустим все тесты (см. рис. 19):

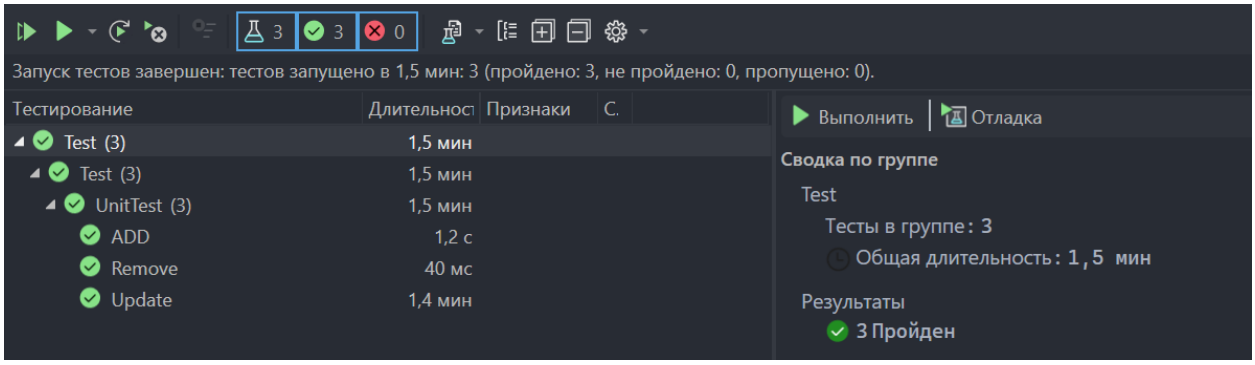


Рисунок 19 – Результаты проведения тестирования

5.4 Разработка структуры документации, приложений к программному обеспечению.

Формирование документации в соответствии с нормативными требованиями

Для понимания работы ПО была разработана кнопка «О программе», которая открывает новое окно с описанием функционала программы (см. рис. 20-22):

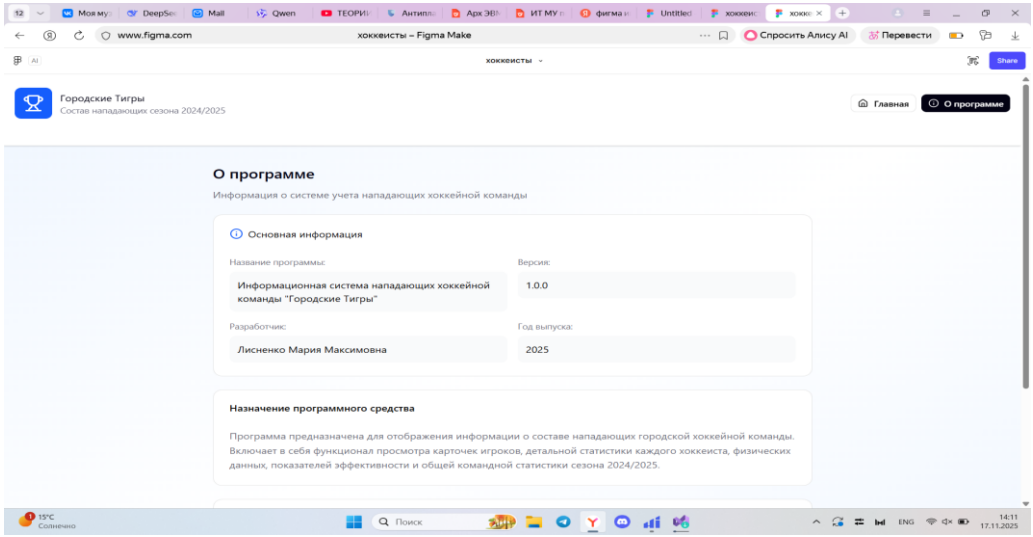


Рисунок 20 – Окно «О программе»

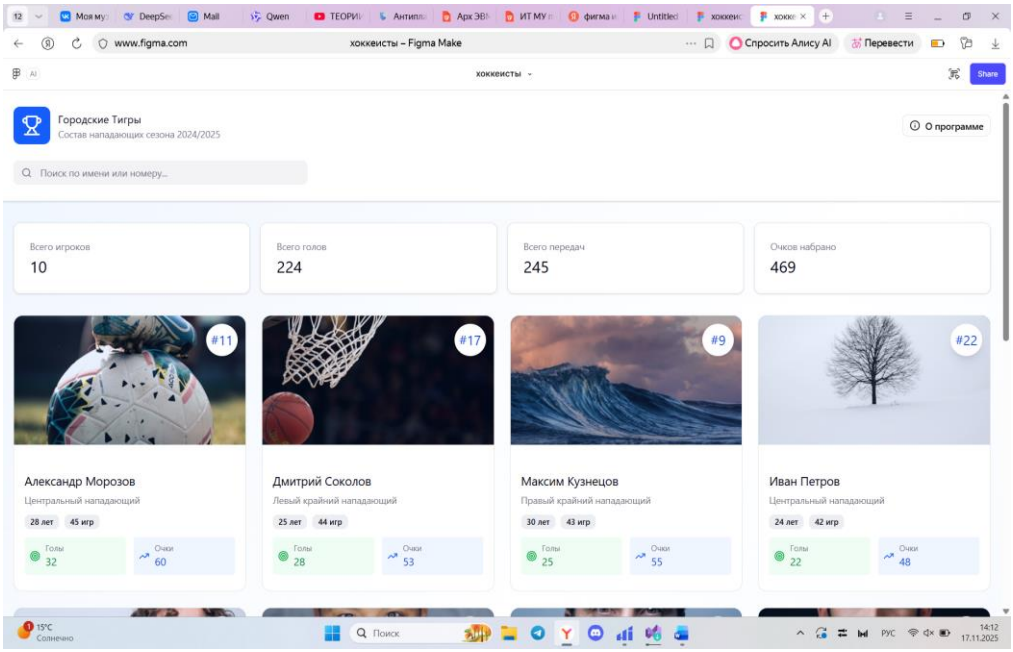


Рисунок 21 – Окно Приложения хоккейной команды «Городские Тигры»

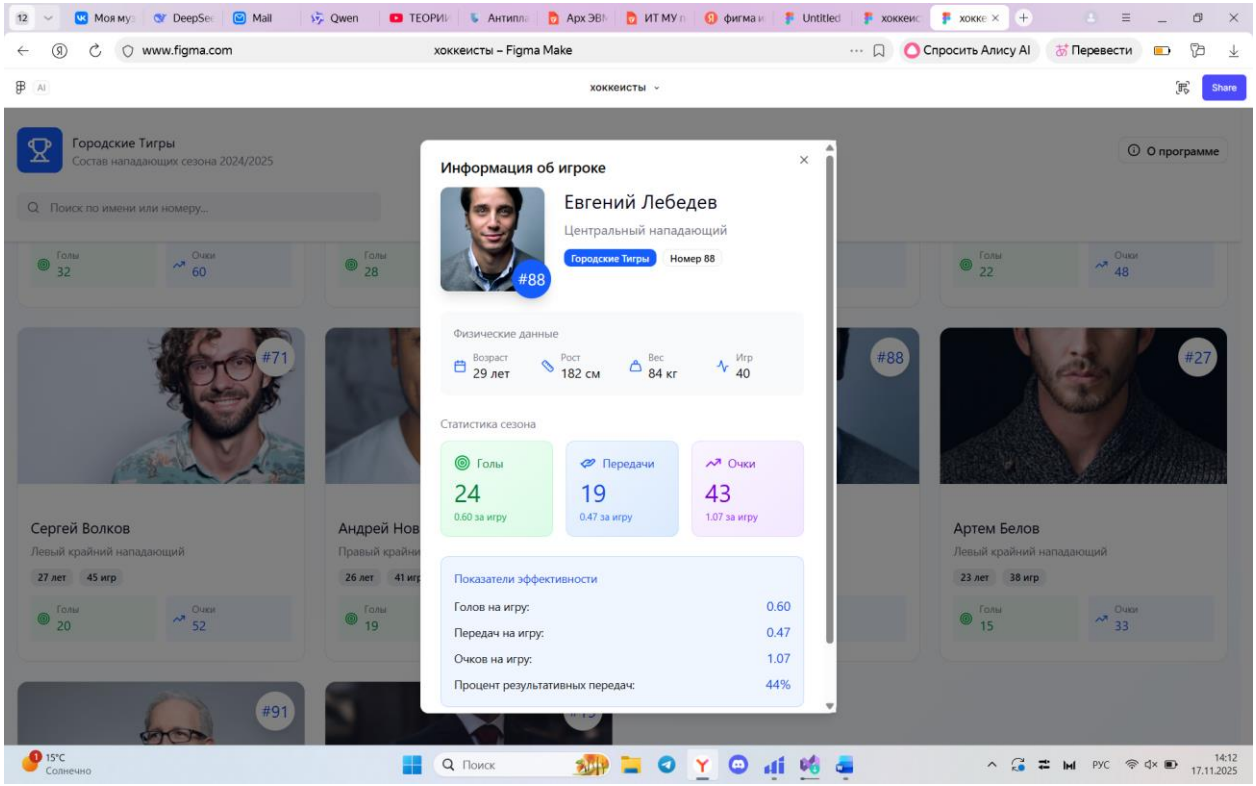


Рисунок 22 – Информация об игроке

Кнопка «О программе» предоставляет пользователям информацию о приложении, включая название, версию, разработчика. Это помогает пользователям понять, кто создал программу, отслеживать обновления и получать поддержку.

Заключение

В данном курсовом проекте были использованы теоретические знания и практические навыки в области разработки программного обеспечения. Проведен анализ осуществимости проекта и разработаны меры по предотвращению потенциальных проблем.

В ходе анализа была подтверждена возможность успешной реализации проекта. Проект направлен на создание программы для управления таблицей, содержащей информацию о хоккеистах-нападающих.

Требования к проекту четко определены и включают создание, управление и анализ таблицы, а также функции по изменению данных, расчету статистики и другие ключевые возможности.

В результате было разработано ПО для сведений о хоккеистах-нападающих с использованием языка программирования C#, а также языка запросов SQL.

Разработанная программа завершена и может быть использована в любом спортивном клубе при необходимости управления составом команды.

Список использованных источников

1. Гагарина Л.Г., Кокорева Е.В., Сидорова-Виснадул Б.Д. Технология разработки программного обеспечения: учеб. пособие; под ред. Л.Г. Гагариной. − М: ИД «ФОРУМ»: ИНФРА-М, 2019. − 400 с.: URL: https://znanium.com/catalog/product/1011120.

2. Документация Microsoft. C#. URL: https://learn.microsoft.com/ru-ru/dotnet/csharp/ (дата обращения: 11.09.2024).

3. Зубкова Т.М. Технология разработки программного обеспечения / Т. М. Зубкова. — 3-е изд., стер. — Санкт-Петербург : Лань, 2023. — 252 с. — URL: https://e.lanbook.com/book/276419

4. Метанит. Entity Framework Core. URL: https://metanit.com/sharp/efcore/ (дата обращения: 27.10.2024).

5. Михайленко К.А., Ковалева К. А. Обзор и анализ развития программного обеспечения // Актуальные проблемы науки и образования в условиях современных вызовов: сборник материалов II Междунар. науч.-практ. конф., Москва, 4 июля 2021 года. – М.: ООО "Институт развития образования и консалтинга", 2021. – С. 52-55.

6. Мякишев Д.В. Разработка программного обеспечения АСУ ТП на основе объектно-ориентированного подхода: метод. пособие – М.; Вологда: Инфра-Инженерия, 2019. − 128 с. − URL: https://znanium.com/catalog/product/1048733.

7. Павловская Т.А. C#. Программирование на языке высокого уровня: Учебник для вузов. – СПб.: Питер, 2014. – 432 с.: ил

8. Федькова Н.А. Современные технологии разработки программного обеспечения: уч.-метод. пособие – Брянск: Брянский ГАУ, 2022. — 58 с. — URL: https://e.lanbook.com/book/305087. (Дата обращения: 13.06.2024).

9. Федькова Н.А. Современные технологии разработки программного обеспечения: уч.-метод. пособие – Брянск : Брянский ГАУ, 2022. — 58 с. — URL: https://e.lanbook.com/book/305087

Приложение А

Листинг 1 – Файл «UnitTest.py»

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using Moq;
using System;
using System.Collections.Generic;
using System.Data.Entity;
using System.Linq;
using WinHockeyManager;

namespace Test
{
    [TestClass]
    public class UnitTest
    {
        private Mock<HockeyManagerDBContext> db;
        private DataService data;

        [TestInitialize]
        public void Setup()
        {
            db = new Mock<HockeyManagerDBContext>();

            var players = new List<HockeyPlayer>{ new HockeyPlayer { PlayerID
= 1, Surname = "Щербинин" } }.AsQueryable();

            var coaches = new List<Coach>{ new Coach { CoachID = 1, Surname =
"Червоткин" } }.AsQueryable();

            var contracts = new List<PlayerContract>{ new PlayerContract {
ContractNumber = 1, Position = "Нападающий", Salary = 15000, PlayerID = 1, CoachID =
1 } }.AsQueryable();

            var mockPlayers = new Mock<DbSet<HockeyPlayer>>();
            var mockCoaches = new Mock<DbSet<Coach>>();
            var mockContracts = new Mock<DbSet<PlayerContract>>();

            mockPlayers.As<IQueryable<HockeyPlayer>>().Setup(m             =>
m.Provider).Returns(players.Provider);
            mockPlayers.As<IQueryable<HockeyPlayer>>().Setup(m             =>
m.Expression).Returns(players.Expression);
            mockPlayers.As<IQueryable<HockeyPlayer>>().Setup(m             =>
m.ElementType).Returns(players.ElementType);
            mockPlayers.As<IQueryable<HockeyPlayer>>().Setup(m             =>
m.GetEnumerator()).Returns(players.GetEnumerator());

            mockCoaches.As<IQueryable<Coach>>().Setup(m                    =>
m.Provider).Returns(coaches.Provider);
            mockCoaches.As<IQueryable<Coach>>().Setup(m                    =>
m.Expression).Returns(coaches.Expression);
            mockCoaches.As<IQueryable<Coach>>().Setup(m                    =>
m.ElementType).Returns(coaches.ElementType);
            mockCoaches.As<IQueryable<Coach>>().Setup(m                    =>
m.GetEnumerator()).Returns(coaches.GetEnumerator());

            mockContracts.As<IQueryable<PlayerContract>>().Setup(m         =>
m.Provider).Returns(contracts.Provider);
```

```
                    mockContracts.As<IQueryable<PlayerContract>>().Setup(m         =>
m.Expression).Returns(contracts.Expression);
                    mockContracts.As<IQueryable<PlayerContract>>().Setup(m         =>
m.ElementType).Returns(contracts.ElementType);
                    mockContracts.As<IQueryable<PlayerContract>>().Setup(m         =>
m.GetEnumerator()).Returns(contracts.GetEnumerator());

                    db.Setup(c => c.Players).Returns(mockPlayers.Object);
                    db.Setup(c => c.Coaches).Returns(mockCoaches.Object);
                    db.Setup(c => c.Contracts).Returns(mockContracts.Object);

                    data = new DataService(db.Object);
                }

                [TestMethod]
                public void ADD()
                {
                    data.ADD("Тиликин",  "Ярыш",  "Нападающий",  20000m,  DateTime.Now,
DateTime.Now.AddYears(2));

                    db.Verify(m      =>      m.Players.Add(It.IsAny<HockeyPlayer>()),
Times.Once);
                    db.Verify(m => m.Coaches.Add(It.IsAny<Coach>()), Times.Once);
                    db.Verify(m    =>    m.Contracts.Add(It.IsAny<PlayerContract>()),
Times.Once);
                    db.Verify(m => m.SaveChanges(), Times.Once);
                }

                [TestMethod]
                public void Remove()
                {
                    data.Remove("Щербинин");

                    db.Verify(m     =>     m.Players.Remove(It.IsAny<HockeyPlayer>()),
Times.Once);
                    db.Verify(m   =>   m.Contracts.Remove(It.IsAny<PlayerContract>()),
Times.Once);
                    db.Verify(m => m.SaveChanges(), Times.Once);
                }

                [TestMethod]
                public void Update()
                {
                    var  contracts  =  new  List<PlayerContract>{  new  PlayerContract  {
ContractNumber = 1, Position = "Центральный нападающий", Salary = 1200m, PlayerID =
1, CoachID = 1 }}.AsQueryable();
                    var mockSetContracts = new Mock<DbSet<PlayerContract>>();
                    mockSetContracts.As<IQueryable<PlayerContract>>().Setup(m      =>
m.Provider).Returns(contracts.Provider);
                    mockSetContracts.As<IQueryable<PlayerContract>>().Setup(m      =>
m.Expression).Returns(contracts.Expression);
                    mockSetContracts.As<IQueryable<PlayerContract>>().Setup(m      =>
m.ElementType).Returns(contracts.ElementType);
                    mockSetContracts.As<IQueryable<PlayerContract>>().Setup(m      =>
m.GetEnumerator()).Returns(contracts.GetEnumerator());

                    db.Setup(c => c.Contracts).Returns(mockSetContracts.Object);

                    data.Update(1,   DateTime.Now.AddDays(1),   "Иванов",   "Петров",
"Крайний нападающий", 1200m, DateTime.Now, DateTime.Now);
```

```
                    var  updatedContract  =  db.Object.Contracts.FirstOrDefault(o  =>
o.ContractNumber == 1);
                    Assert.IsNotNull(updatedContract);
                    Assert.AreEqual(1200m, updatedContract.Salary);
                    Assert.AreEqual("Центральный                        нападающий",
updatedContract.Position);
                }
            }
        }
```

Листинг 2 – Программная реализация окна «О программе»

```
using MetroFramework.Forms;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace WinTailorPro
{
    public partial class FormAboutTheProgram : MetroForm
    {
        public FormAboutTheProgram()
        {
            InitializeComponent();
            // Убрали чтение из файла и используем текст про хоккеистов
            SetHockeyText();
        }

        private void SetHockeyText()
        {
            string hockeyText = @"Хоккей - это динамичная и захватывающая спортивная
игра, в которой две команды, используя клюшки, пытаются забросить шайбу в ворота
соперника.

Основные особенности хоккея:
• Играется на ледовой площадке
• В каждой команде по 6 игроков (5 полевых + вратарь)
• Игра состоит из трех периодов по 20 минут
• Физическая контактность разрешена в рамках правил

Хоккей требует от игроков:
- Отличной физической подготовки
- Координации и ловкости
- Командного взаимодействия
- Быстрой реакции и стратегического мышления

Это один из самых популярных видов спорта в многих странах мира, особенно в России,
Канаде, США и странах Северной Европы.";

            tBox.Text = hockeyText;
        }
        // Старый метод можно удалить или закомментировать
        /*
```

```
        private void ReadFromFile()
        {
            StreamReader file = new StreamReader("ОПрограмме.txt");
            while (!file.EndOfStream)
            {
                string str = file.ReadLine();
                tBox.Text = str;
            }
            file.Close();
        }
        */
    }
}
```